

0.1 Estensioni, specie di agenti e variabili principali del modello

```
1 extensions [ fetch table csv ]
2
3 breed [ agents-a agent-a ]
4 breed [ agents-b agent-b ]
5 breed [ agents-c agent-c ]
6
7 globals [
8   prices
9   day-index
10
11   ;; DATI REALI (STARTER PER LA CORRENTE)
12   btc-price
13   btc-initial-price
14   btc-ath-price
15   btc-prev-price
16   btc-return
17   btc-volatility
18
19   ;; PREZZO E VOLATILITA' STIMATI (ENDOGENI)
20   model-price
21   model-return
22   model-volatility
23   model-trading-volume
24   real-trading-volume-usd
25
26   ;; PARAMETRI CORRENTE STARTER E SWITCH DINAMICO
27   current-disabled?
28   ;; pearson-threshold viene gestito dallo slider nell'interfaccia
29   current-hype                ;; Livello di stress/hype di mercato
30
31   ;; SCALING E ISTERESI VOLUMI
32   real-volume-initial
33   real-volume-ath
34   volume-hysteresis-state
35
36   ;; STORICO E CORRELAZIONE VOLUMI (PEARSON)
37   model-volume-history
38   real-volume-history
39   volume-correlation
40   volume-correlation-history
41   mean-volume-correlation
42   above-threshold-counter    ;; Conta i tick consecutivi sopra pearson-threshold
43
44   ;; STORICO E CORRELAZIONE PREZZI (PEARSON)
45   model-price-history
46   real-price-history
47   price-correlation
48   price-correlation-history
49   mean-price-correlation
50
51   op-history
52   ya-history
53   yb-history
54   lambda-instability
55   heterogeneity-index-value
56   phase-correlation-ab
57 ]
58
59 turtles-own [
60   flockmates
61   bitcoin-held
62   cash-balance
63   velocity-x
```

```

64 velocity-y
65 ]

```

Listing 1: Dichiarazione delle estensioni, delle specie e delle variabili principali

Questa prima parte del codice definisce la struttura generale del modello. In particolare, vengono dichiarate le estensioni utilizzate, le specie di agenti e le variabili globali o individuali che saranno impiegate nelle procedure successive.

La riga

```
extensions [ fetch table csv ]
```

carica tre estensioni di NetLogo. L'estensione **fetch** permette di effettuare richieste verso risorse esterne, ad esempio scaricando dati da una API. In questo modello essa è utile per ottenere dati reali relativi a Bitcoin. L'estensione **table** consente di manipolare strutture dati del tipo chiave-valore, particolarmente utili quando si ricevono dati in formato strutturato. Infine, l'estensione **csv** permette di leggere o scrivere dati in formato CSV, cioè in un formato tabellare comunemente usato per salvare serie storiche o risultati di simulazione. Le tre righe successive definiscono tre specie di agenti tramite il costrutto **breed**:

```
breed [ agents-a agent-a ],    breed [ agents-b agent-b ],    breed [ agents-c agent-c ].
```

In NetLogo, una **breed** rappresenta una sottopopolazione di turtles. Il primo nome all'interno della parentesi quadra è il nome plurale della specie, mentre il secondo è il nome singolare. Ad esempio, **agents-a** indica l'insieme degli agenti della prima specie, mentre **agent-a** indica un singolo agente di quella specie.

L'introduzione di più specie permette di distinguere gruppi differenti di agenti. Dal punto di vista modellistico, tali gruppi possono rappresentare categorie diverse di operatori, strategie differenti, popolazioni eterogenee oppure classi di agenti con comportamenti o parametri differenti. Anche se appartengono a specie diverse, tutti questi agenti rimangono comunque turtles: un comando come

```
ask turtles [ ... ]
```

agisce quindi su tutte le specie contemporaneamente.

Il blocco **globals** contiene le variabili globali del modello. Una variabile globale non appartiene a un singolo agente, ma descrive una proprietà dell'intero sistema o una grandezza accessibile da tutte le procedure.

Le prime due variabili sono

```
prices,    day-index.
```

La variabile **prices** contiene la serie storica dei prezzi scaricati o letti dal modello. La variabile **day-index** indica invece quale elemento della serie storica viene utilizzato al passo corrente della simulazione. Essa funziona quindi come indice temporale: aumentando **day-index**, il modello procede lungo la sequenza dei dati di mercato.

La sezione successiva riguarda i dati reali di Bitcoin:

```
btc-price,  btc-initial-price,  btc-ath-price,  btc-prev-price,  btc-return,  btc-volatility.
```

Bitcoin è una criptovaluta, cioè un'attività digitale scambiata su mercati elettronici. Come per un'azione o per un altro strumento finanziario, il suo prezzo varia nel tempo in base all'incontro tra domanda e offerta. La variabile **btc-price** rappresenta il prezzo reale corrente di Bitcoin nel dato temporale considerato. La variabile **btc-initial-price** memorizza il prezzo iniziale della serie utilizzata, mentre **btc-ath-price** rappresenta il massimo storico raggiunto nella finestra di dati considerata. La sigla ATH significa *All-Time High*, cioè massimo storico o massimo raggiunto da una grandezza in un certo intervallo di osservazione.

La variabile **btc-prev-price** contiene il prezzo precedente di Bitcoin ed è necessaria per calcolare il rendimento. In finanza, il rendimento misura la variazione relativa del prezzo tra due istanti consecutivi. Se $P(t)$ indica il prezzo corrente e $P(t-1)$ il prezzo precedente, il rendimento può essere definito come

$$r(t) = \frac{P(t) - P(t-1)}{P(t-1)}.$$

Nel modello, il rendimento reale di Bitcoin viene memorizzato nella variabile

```
btc-return.
```

La variabile **btc-volatility** rappresenta invece la volatilità reale di Bitcoin. In ambito finanziario, la volatilità misura l'intensità delle variazioni di prezzo. Un mercato molto volatile è un mercato in cui il prezzo cambia molto rapidamente o con variazioni ampie. In una versione semplificata, la volatilità può essere calcolata come valore assoluto del rendimento:

$$V(t) = |r(t)|.$$

Questa scelta elimina il segno della variazione: una forte crescita e una forte diminuzione del prezzo producono entrambe un valore elevato di volatilità.

Il blocco successivo introduce le variabili endogene del modello:

model-price, model-return, model-volatility, model-trading-volume, real-trading-volume-usd.

Una variabile si dice endogena quando viene prodotta internamente dal modello. In questo caso, **model-price** rappresenta il prezzo generato dalla dinamica simulata degli agenti, mentre **btc-price** rappresenta un prezzo reale esterno. La distinzione tra prezzo reale e prezzo del modello è importante: il primo deriva dai dati empirici, il secondo deriva invece dall'interazione tra gli agenti.

La variabile **model-return** rappresenta il rendimento del prezzo endogeno, calcolato confrontando il prezzo prodotto dal modello con il suo valore precedente. Analogamente, **model-volatility** misura l'intensità delle variazioni del prezzo simulato.

La variabile **model-trading-volume** rappresenta il volume di scambio generato dagli agenti del modello. In finanza, il volume di scambio indica la quantità complessiva di un'attività finanziaria comprata o venduta in un certo intervallo di tempo. Nel caso del modello, il volume non deriva dal mercato reale, ma dal numero di operazioni effettuate dagli agenti.

La variabile **real-trading-volume-usd** rappresenta invece il volume reale del mercato, espresso in dollari. Il suffisso **usd** indica che il valore è misurato in dollari statunitensi. Questa variabile permette di confrontare il volume simulato con il volume reale osservato.

La sezione dedicata alla corrente e allo switch dinamico contiene le variabili

current-disabled?, current-hype.

La variabile **current-disabled?** è una variabile booleana, cioè una variabile che può assumere valore vero o falso. Il punto interrogativo finale è una convenzione frequente in NetLogo per indicare variabili o reporter logici. Essa serve a stabilire se la corrente esterna deve essere disattivata.

La corrente rappresenta una spinta o un campo esterno che influenza il movimento degli agenti. Nelle prime fasi del modello, questa corrente può essere guidata dai dati reali del mercato, ad esempio dalla volatilità di Bitcoin. In seguito, il modello può decidere di disattivare o modificare questa dipendenza esterna in base alla qualità dell'accordo tra il mercato reale e il mercato simulato.

La variabile **current-hype** rappresenta il livello di stress o hype di mercato. Dal punto di vista economico, il termine *hype* indica una fase di forte attenzione, entusiasmo o agitazione collettiva intorno a un'attività finanziaria. In un mercato con alto hype, gli agenti possono essere più reattivi, più propensi a operare e più sensibili alle variazioni di prezzo o volume.

Il commento relativo a **pearson-threshold** indica che tale soglia non è dichiarata come variabile globale nel codice, perché viene gestita tramite uno slider dell'interfaccia. Essa rappresenta una soglia utilizzata per stabilire se la correlazione tra dati reali e dati simulati è sufficientemente elevata.

La sezione successiva riguarda lo scaling e l'isteresi dei volumi:

real-volume-initial, real-volume-ath, volume-hysteresis-state.

La variabile **real-volume-initial** memorizza il volume reale iniziale, mentre **real-volume-ath** memorizza il massimo volume reale osservato nella finestra di dati considerata. Anche qui, **ath** indica un massimo storico o un massimo relativo all'intervallo analizzato.

Il concetto di scaling indica una trasformazione usata per portare grandezze di ordini di grandezza diversi su scale confrontabili. Questo è utile perché il volume reale di mercato può essere enormemente più grande del volume prodotto da un numero limitato di agenti simulati.

La variabile **volume-hysteresis-state** rappresenta lo stato di isteresi dei volumi. In generale, si parla di isteresi quando lo stato attuale di un sistema non dipende solo dal valore corrente di una variabile, ma anche dalla sua storia precedente. Dal punto di vista economico, ciò può rappresentare un effetto di memoria del mercato: un periodo di forte volume o forte stress può continuare a influenzare il comportamento degli agenti anche dopo che il segnale originario si è ridotto.

Le variabili successive memorizzano lo storico dei volumi e la loro correlazione:

`model-volume-history`, `real-volume-history`, `volume-correlation`, `volume-correlation-history`, `mean-vol`

Le variabili `model-volume-history` e `real-volume-history` contengono rispettivamente la sequenza temporale dei volumi prodotti dal modello e dei volumi reali. Questi storici sono necessari per confrontare la dinamica simulata con quella empirica.

La variabile `volume-correlation` contiene il coefficiente di correlazione di Pearson tra i volumi del modello e i volumi reali. La correlazione di Pearson misura il grado di relazione lineare tra due serie di dati. Se X e Y sono due serie temporali, il coefficiente di Pearson può essere scritto come

$$\rho_{X,Y} = \frac{\sum_{i=1}^N (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^N (X_i - \bar{X})^2} \sqrt{\sum_{i=1}^N (Y_i - \bar{Y})^2}}.$$

Il valore di ρ appartiene all'intervallo $[-1, 1]$. Se ρ è vicino a 1, le due serie tendono a muoversi nella stessa direzione. Se ρ è vicino a -1 , tendono a muoversi in direzioni opposte. Se ρ è vicino a 0, non emerge una relazione lineare significativa.

La variabile `volume-correlation-history` conserva lo storico dei valori di correlazione calcolati nel tempo, mentre `mean-volume-correlation` memorizza una correlazione media. Infine, `above-threshold-counter` conta per quanti tick consecutivi la correlazione rimane sopra la soglia `pearson-threshold`. Questa variabile è utile per evitare decisioni basate su oscillazioni momentanee: il modello può richiedere che la correlazione rimanga alta per più passi temporali prima di modificare il proprio regime dinamico.

Una struttura analoga viene usata per i prezzi:

`model-price-history`, `real-price-history`, `price-correlation`, `price-correlation-history`, `mean-price-c`

In questo caso il confronto non riguarda i volumi, ma i prezzi. La variabile `model-price-history` conserva la serie storica dei prezzi endogeni prodotti dal modello, mentre `real-price-history` conserva la serie dei prezzi reali di Bitcoin. La variabile `price-correlation` misura la correlazione di Pearson tra queste due serie.

Il confronto tra prezzo simulato e prezzo reale permette di valutare se la dinamica interna del modello riesce a riprodurre, almeno in parte, l'andamento osservato nel mercato reale. Anche in questo caso è importante ricordare che una correlazione elevata non implica causalità: indica soltanto che le due serie tendono a muoversi in modo linearmente simile.

Le ultime variabili globali sono

`op-history`, `ya-history`, `yb-history`, `lambda-instability`, `heterogeneity-index-value`, `phase-correlati`

La variabile `op-history` può essere interpretata come lo storico del parametro d'ordine. Il parametro d'ordine misura il grado di allineamento collettivo degli agenti: valori vicini a 1 indicano un sistema molto allineato, mentre valori vicini a 0 indicano un sistema disordinato.

Le variabili `ya-history` e `yb-history` memorizzano storici legati alle coordinate verticali o alle dinamiche medie di specifiche specie di agenti. Poiché nel modello la coordinata y può essere associata alla propensione ad acquistare o vendere, questi storici possono essere utilizzati per studiare l'evoluzione comportamentale dei gruppi.

La variabile `lambda-instability` rappresenta una misura di instabilità del sistema. Il simbolo λ viene spesso usato per indicare un parametro che misura l'intensità di una crescita, di una divergenza o di una sensibilità dinamica. Nel contesto del modello, questa variabile può essere usata per quantificare quanto il sistema sia stabile o instabile nel tempo.

La variabile `heterogeneity-index-value` rappresenta un indice di eterogeneità. L'eterogeneità misura quanto gli agenti o i gruppi differiscono tra loro. Un valore elevato può indicare che specie diverse assumono comportamenti molto differenti, mentre un valore basso può indicare una maggiore omogeneità della popolazione.

Infine, `phase-correlation-ab` misura una correlazione di fase tra le specie `agents-a` e `agents-b`. In termini qualitativi, questa grandezza può indicare se due gruppi oscillano o si muovono in modo coordinato, oppure se seguono dinamiche sfasate.

Il blocco `turtles-own` dichiara invece le variabili individuali degli agenti:

`flockmates`, `bitcoin-held`, `cash-balance`, `velocity-x`, `velocity-y`.

A differenza delle variabili globali, queste variabili appartengono separatamente a ciascun agente. Ogni turtle possiede quindi un proprio valore di **bitcoin-held**, un proprio valore di **cash-balance** e così via.

La variabile **flockmates** contiene l'insieme degli agenti vicini con cui la turtle interagisce nella regola di flocking. Essa è fondamentale per l'allineamento locale, perché permette a ogni agente di individuare i vicini rispetto ai quali aggiornare la propria direzione.

La variabile **bitcoin-held** indica quanti Bitcoin sono posseduti da un agente. Essa rappresenta quindi la posizione dell'agente sull'attività finanziaria. La variabile **cash-balance** rappresenta invece la liquidità disponibile, cioè la quantità di denaro che l'agente può usare per acquistare Bitcoin

0.2 Helper per la gestione sicura delle penne dei plot

```
1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; HELPER PER PLOT SICURI
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to assicura-penna [ nome-penna colore ]
6    carefully [
7      set-current-plot-pen nome-penna
8    ] [
9      create-temporary-plot-pen nome-penna
10     set-plot-pen-color colore
11   ]
12 end
```

Listing 2: Procedura di controllo e creazione delle penne dei plot

La procedura **assicura-penna** è una procedura di supporto pensata per rendere più robusta la gestione dei grafici. Essa riceve due argomenti:

nome-penna, colore.

Il primo indica il nome della penna del plot che si vuole utilizzare, mentre il secondo indica il colore da assegnare alla penna nel caso in cui questa debba essere creata.

Il blocco principale è costruito tramite il comando

carefully [...] [...].

In NetLogo, **carefully** permette di provare a eseguire un blocco di codice e, se questo genera un errore, eseguire un secondo blocco alternativo. In questo caso, il modello prova prima a selezionare una penna già esistente:

set-current-plot-pen nome-penna.

Se la penna esiste, viene semplicemente selezionata. Se invece non esiste, NetLogo genererebbe normalmente un errore. Grazie a **carefully**, tale errore viene intercettato e viene eseguito il blocco alternativo:

create-temporary-plot-pen nome-penna

seguito da

set-plot-pen-color colore.

In questo modo, se la penna richiesta non è già presente nel plot, viene creata automaticamente e le viene assegnato il colore specificato.

Questa procedura è utile quando il modello aggiorna molti grafici o molte curve diverse. Invece di dover creare manualmente tutte le penne dall'interfaccia di NetLogo, il codice controlla automaticamente se una penna esiste e, se manca, la crea. Questo rende i plot più flessibili e riduce il rischio di errori dovuti a nomi di penne non ancora definiti.

0.3 Inizializzazione del portafoglio degli agenti

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; INIZIALIZZAZIONE PORTAFOGLIO
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to inizializza-portafoglio
6   set bitcoin-held random 6
7   set cash-balance 50000 + random 1450001
8 end
```

Listing 3: Procedura di inizializzazione del portafoglio individuale

La procedura **inizializza-portafoglio** assegna a ogni agente una dotazione finanziaria iniziale. Essa viene normalmente richiamata durante la creazione degli agenti nella procedura **setup**.

La prima istruzione è

set bitcoin-held random 6.

La variabile **bitcoin-held** rappresenta il numero di Bitcoin posseduti dall'agente. Il comando **random 6** genera un numero intero casuale tra 0 e 5. Di conseguenza, ogni agente inizia la simulazione con una quantità casuale di Bitcoin compresa nell'intervallo

[0, 5].

La seconda istruzione è

set cash-balance 50000 + random 1450001.

La variabile **cash-balance** rappresenta la liquidità disponibile dell'agente, cioè la quantità di denaro che può essere usata per effettuare acquisti. Il termine **random 1450001** genera un numero intero casuale tra 0 e 1 450 000. Sommando 50 000, si ottiene una liquidità iniziale compresa tra

50 000 e 1 500 000.

Questa inizializzazione introduce eterogeneità economica tra gli agenti: alcuni partono con molti Bitcoin e molta liquidità, altri con meno disponibilità.

Dal punto di vista economico, il portafoglio di un agente è composto da due elementi: una parte detenuta in Bitcoin e una parte detenuta in liquidità. La liquidità indica la capacità immediata di acquistare nuovi Bitcoin, mentre **bitcoin-held** indica la quantità dell'attività finanziaria già posseduta. Questa distinzione è necessaria per impedire operazioni non realistiche, come comprare senza denaro disponibile o vendere Bitcoin non posseduti.

0.4 Posizionamento iniziale degli agenti in una regione centrale

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; SPAWN CIRCOLARE CENTRALE
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to posiziona-al-centro
6   let angolo random-float 360
7   let raggio random-float spawn-radius
8
9   setxy (raggio * cos angolo)
10         (raggio * sin angolo)
11 end
```

Listing 4: Procedura di spawn circolare centrale

La procedura **posiziona-al-centro** assegna a ogni agente una posizione iniziale casuale all'interno di una regione circolare centrata nell'origine del mondo NetLogo. Essa viene utilizzata al momento della creazione degli agenti, in modo che la popolazione non venga distribuita casualmente in tutto lo spazio, ma nasca inizialmente in una zona centrale controllata.

La variabile locale

angolo

viene definita tramite

```
let angolo random-float 360.
```

Il comando `random-float 360` genera un numero reale casuale compreso tra 0 e 360. Questo valore rappresenta l'angolo polare della posizione iniziale dell'agente. In altre parole, stabilisce la direzione, rispetto all'origine, lungo la quale verrà collocato l'agente.

La variabile locale

raggio

viene invece definita tramite

```
let raggio random-float spawn-radius.
```

Il parametro `spawn-radius` controlla la dimensione della regione iniziale di spawn. Il comando `random-float spawn-radius` genera un raggio casuale compreso tra 0 e `spawn-radius`. Quindi gli agenti vengono collocati entro un disco di raggio massimo `spawn-radius`.

La posizione cartesiana viene poi ottenuta a partire dalle coordinate polari:

$$x = r \cos(\theta), \quad y = r \sin(\theta).$$

Nel codice NetLogo questa trasformazione è implementata da

```
setxy (raggio * cos angolo) (raggio * sin angolo).
```

Il comando `setxy` assegna simultaneamente la coordinata x e la coordinata y dell'agente. Poiché il centro del disco è l'origine, non è necessario aggiungere alcuna traslazione alle coordinate.

Dal punto di vista modellistico, questa procedura permette di studiare l'evoluzione di un gruppo inizialmente concentrato. Gli agenti partono da una regione comune e successivamente si disperdono, si allineano o si separano in base alle regole di flocking, repulsione, corrente e trading definite nel modello.

0.5 Procedura di inizializzazione del modello: setup

```
1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; SETUP
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to setup
6
7      clear-all
8
9      carefully [
10         import-drawing "grafico_buy_sell.png"
11     ] [ ]
12
13     ;; GARANZIA SOGLIA PEARSON VALIDA DA SLIDER
14     if not is-number? pearson-threshold [
15         set pearson-threshold 0.5
16     ]
17
18     set current-disabled? false      ;; Inizia sempre in Fase 1
19     set current-hype 0
20
21     if is-number? price-sensitivity = false [
22         set price-sensitivity 0.01
23     ]
24
25     set day-index 0
26     set btc-prev-price 0
27     set btc-price 0
28     set btc-initial-price 0
```

```

29  set btc-ath-price 0
30  set btc-return 0
31  set btc-volatility 0
32
33  set model-price 0
34  set model-return 0
35  set model-volatility 0
36
37  set model-trading-volume 0
38  set real-trading-volume-usd 0
39  set real-volume-initial 0
40  set real-volume-ath 0
41  set volume-hysteresis-state 0
42
43  ;; INIZIALIZZAZIONE CORRELAZIONE VOLUMI
44  set volume-correlation 0
45  set volume-correlation-history []
46  set mean-volume-correlation 0
47  set above-threshold-counter 0
48
49  ;; INIZIALIZZAZIONE CORRELAZIONE PREZZI
50  set model-price-history []
51  set real-price-history []
52  set price-correlation 0
53  set price-correlation-history []
54  set mean-price-correlation 0
55
56  set prices []
57  set model-volume-history []
58  set real-volume-history []
59
60  set op-history []
61  set ya-history []
62  set yb-history []
63
64  set lambda-instability 0
65  set heterogeneity-index-value 0
66  set phase-correlation-ab 0
67
68  if trade-interval <= 0 [
69      user-message "Attenzione: trade-interval deve essere > 0. Impostato a 1."
70      set trade-interval 1
71  ]
72
73  scarica-dati
74
75  ;; CREAZIONE AGENTI
76  create-agents-a population-a [
77      set color yellow
78      set size 10
79      posiziona-al-centro
80      set heading random 360
81      set velocity-x cos heading
82      set velocity-y sin heading
83      set flockmates no-turtles
84      inizializza-portafoglio
85  ]
86
87  create-agents-b population-b [
88      set color red
89      set size 10
90      posiziona-al-centro
91      set heading random 360
92      set velocity-x cos heading
93      set velocity-y sin heading

```

```

94     set flockmates no-turtles
95     inizializza-portafoglio
96 ]
97
98 create-agents-c population-c [
99     set color cyan
100    set size 10
101    posiziona-al-centro
102    set heading random 360
103    set velocity-x cos heading
104    set velocity-y sin heading
105    set flockmates no-turtles
106    inizializza-portafoglio
107 ]
108
109 ;; RETE FISSA
110 ask agents-a [
111     let n min (list number-of-followers (count other agents-a))
112     if n > 0 [ set flockmates n-of n other agents-a ]
113 ]
114
115 ask agents-b [
116     let n min (list number-of-followers (count other agents-b))
117     if n > 0 [ set flockmates n-of n other agents-b ]
118 ]
119
120 ask agents-c [
121     let n min (list number-of-followers (count other agents-c))
122     if n > 0 [ set flockmates n-of n other agents-c ]
123 ]
124
125 ;; INIZIALIZZAZIONE PRIMO GIORNO REALE E MODELLO
126 if is-list? prices and length prices > 0 [
127     let record item 0 prices
128     set btc-price item 1 record
129     set btc-prev-price btc-price
130     set btc-initial-price btc-price
131     set btc-ath-price btc-price
132     set day-index 1
133 ]
134
135 set model-price btc-price
136 set model-return 0
137 set model-volatility 0
138
139 print (word "=== SETUP COMPLETATO ===")
140 print (word "FASE 1 ATTIVA - Switch automatico a Fase 2 quando r_medio(volumi) >=
    " pearson-threshold " per piu' di 5 tick consecutivi.")
141
142 reset-ticks
143
144 end

```

Listing 5: Procedura di setup del modello

La procedura `setup` inizializza l'intero modello e prepara sia la parte finanziaria sia la parte agent-based della simulazione. Il comando `clear-all` elimina lo stato precedente del modello, cancellando agenti, plot, variabili temporanee e configurazioni residue. In questo modo ogni nuova esecuzione parte da una condizione pulita. Subito dopo, il codice prova a importare l'immagine `grafico_buy_sell.png` tramite il blocco

```
carefully [ import-drawing ... ] [ ]
```

Il comando `carefully` permette di evitare che il modello si blocchi nel caso in cui il file immagine non sia presente. Se l'importazione fallisce, il secondo blocco vuoto viene eseguito e la simulazione continua comunque. La parte successiva controlla che alcuni parametri dell'interfaccia abbiano valori numerici validi. In particolare, `pearson-threshold` viene impostato a 0.5 se non è un numero. Questa variabile rappresenta la soglia di

correlazione di Pearson richiesta per considerare sufficientemente simili due serie temporali, ad esempio i volumi reali e i volumi generati dal modello. Analogamente, `price-sensitivity` viene impostata a 0.01 se non è definita correttamente. Tale parametro controlla quanto il prezzo endogeno del modello reagisce agli squilibri generati dagli agenti.

La variabile `current-disabled?` viene inizializzata a `false`. Questo indica che, all'inizio, la corrente esterna è attiva. Il commento specifica che il modello parte sempre dalla *Fase 1*, cioè da una fase in cui il sistema è ancora guidato, almeno in parte, dai dati reali. La variabile `current-hype` viene posta a zero e rappresenta il livello iniziale di hype o stress di mercato.

Segue l'inizializzazione delle variabili relative al mercato reale di Bitcoin: `btc-price`, `btc-prev-price`, `btc-initial-price`, `btc-ath-price`, `btc-return` e `btc-volatility`. Il prezzo di Bitcoin rappresenta il valore dell'attività finanziaria nel tempo; il rendimento misura la variazione relativa del prezzo tra due istanti consecutivi; la volatilità misura invece l'intensità delle variazioni di prezzo. Il valore `btc-ath-price` memorizza il massimo prezzo raggiunto nella finestra di osservazione considerata.

Vengono poi inizializzate le variabili endogene del modello: `model-price`, `model-return` e `model-volatility`. A differenza delle variabili precedenti, queste non provengono direttamente dai dati reali, ma sono generate dalla dinamica interna della simulazione. Il prezzo endogeno è quindi il prezzo prodotto dal comportamento degli agenti.

La procedura azzerava anche le variabili relative ai volumi: `model-trading-volume`, `real-trading-volume-usd`, `real-volume-initial`, `real-volume-ath` e `volume-hysteresis-state`. In economia, il volume di scambio misura la quantità o il controvalore di un'attività comprata e venduta in un certo intervallo di tempo. Nel modello si distinguono i volumi reali, provenienti dal mercato, e i volumi endogeni, generati dagli scambi degli agenti.

Le liste storiche e le variabili di correlazione vengono inizializzate come liste vuote o valori nulli. In particolare, `model-volume-history` e `real-volume-history` conterranno gli storici dei volumi simulati e reali, mentre `model-price-history` e `real-price-history` conterranno gli storici dei prezzi. Le variabili `volume-correlation` e `price-correlation` misureranno la correlazione di Pearson tra le serie simulate e quelle reali. Il contatore `above-threshold-counter` serve a verificare per quanti tick consecutivi la correlazione dei volumi rimane sopra la soglia fissata.

La procedura inizializza anche alcune grandezze dinamiche legate al comportamento collettivo degli agenti: `op-history`, `ya-history`, `yb-history`, `lambda-instability`, `heterogeneity-index-value` e `phase-correlation-ab`. Queste variabili permettono di studiare il grado di ordine, l'instabilità, l'eterogeneità tra gruppi e la relazione di fase tra specie diverse.

Prima di scaricare i dati, il codice controlla che `trade-interval` sia positivo. Questo parametro stabilisce ogni quante iterazioni avviene il trading. Se il valore è minore o uguale a zero, NetLogo mostra un messaggio di avviso e imposta automaticamente `trade-interval` a 1, evitando divisioni o condizioni temporali non valide.

Il comando

`scarica-dati`

richiama la procedura che carica la serie dei prezzi reali di Bitcoin. Dopo il download, il modello crea le tre popolazioni di agenti: `agents-a`, `agents-b` e `agents-c`. Le tre specie vengono distinte tramite il colore: giallo, rosso e ciano. Ogni agente viene posizionato nella regione centrale tramite `posiziona-al-centro`, riceve una direzione iniziale casuale e le corrispondenti componenti di velocità:

$$v_x = \cos(\theta), \quad v_y = \sin(\theta),$$

dove θ è il valore di `heading`. Ogni agente viene inoltre inizializzato senza vicini di flocking e riceve un portafoglio iniziale tramite `inizializza-portafoglio`.

La sezione denominata *rete fissa* assegna a ciascun agente un insieme iniziale di `flockmates` appartenenti alla stessa specie. Per ogni gruppo, il numero di vicini scelti è il minimo tra `number-of-followers` e il numero effettivo di altri agenti disponibili nella stessa specie. Il comando `n-of` seleziona casualmente n agenti. In questo modo il flocking non dipende necessariamente da una vicinanza spaziale istantanea, ma da una rete di relazioni interne alla specie.

Infine, se la lista `prices` contiene dati validi, il primo record viene usato per inizializzare il primo prezzo reale di Bitcoin. Il prezzo corrente, il prezzo precedente, il prezzo iniziale e il massimo iniziale vengono tutti posti uguali al primo prezzo disponibile. L'indice temporale `day-index` viene poi impostato a 1, così che l'aggiornamento successivo legga il record seguente.

Il prezzo endogeno `model-price` viene inizializzato uguale al prezzo reale iniziale. Questa scelta permette al modello simulato di partire dallo stesso livello del mercato reale, rendendo più significativo il confronto successivo tra prezzo reale e prezzo generato dagli agenti.

La procedura termina stampando alcuni messaggi informativi nella console e richiamando `reset-ticks`, che azzera il tempo della simulazione.

0.6 Hype di mercato e stubbornness dinamica

```

1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; HYPE E STUBBORNNESS DINAMICA
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to-report calcola-hype
6    report (abs model-return) + model-volatility
7  end
8
9  to-report dynamic-stubbornness
10   let base-s 0.95
11   let min-s 0.05
12   let raw-s base-s - (hype-sensitivity * current-hype)
13   report max (list min-s (min (list base-s raw-s)))
14 end

```

Listing 6: Calcolo dell'hype e della stubbornness dinamica

Questo blocco introduce due reporter legati alla reattività degli agenti rispetto alle condizioni del mercato simulato. Il primo reporter, `calcola-hype`, definisce una misura sintetica dell'hype o stress di mercato:

$$H(t) = |r_m(t)| + \sigma_m(t),$$

dove $r_m(t)$ è il rendimento del prezzo endogeno del modello e $\sigma_m(t)$ è la volatilità endogena. Il termine $|r_m(t)|$ misura l'intensità della variazione del prezzo, indipendentemente dal segno, mentre `model-volatility` rappresenta il grado di instabilità del prezzo simulato.

Dal punto di vista economico, l'hype può essere interpretato come una misura aggregata della tensione del mercato. Quando il prezzo cambia molto e la volatilità è elevata, il mercato viene considerato più instabile, più reattivo o più emotivamente carico. In questo senso, `current-hype` rappresenta una variabile comportamentale che traduce l'instabilità del prezzo in una maggiore sensibilità degli agenti.

Il secondo reporter, `dynamic-stubbornness`, calcola una stubbornness dinamica, cioè una misura della tendenza degli agenti a conservare il proprio orientamento o comportamento precedente. Il valore di partenza è

$$s_{\text{base}} = 0.95,$$

mentre il valore minimo ammesso è

$$s_{\text{min}} = 0.05.$$

La quantità intermedia

$$s_{\text{raw}} = s_{\text{base}} - \text{hype-sensitivity} \cdot \text{current-hype}$$

riduce la stubbornness quando aumenta l'hype di mercato. Quindi, in condizioni di mercato calme, gli agenti rimangono più stabili e meno reattivi; in condizioni di mercato stressato, diventano più sensibili e più disposti a modificare il loro stato.

L'ultima riga,

$$\text{report max (list min-s (min (list base-s raw-s))),}$$

impone che la stubbornness resti sempre compresa tra `min-s` e `base-s`. In forma matematica:

$$s(t) = \max[s_{\text{min}}, \min(s_{\text{base}}, s_{\text{raw}}(t))].$$

Questa limitazione evita valori non realistici: la stubbornness non può superare il valore massimo iniziale e non può scendere sotto una soglia minima.

0.7 Pipeline dinamica della simulazione

```
1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; GO (PIPELINE)
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to go
6
7    set model-trading-volume 0
8
9    ;; 1. AGGIORNAMENTO PREZZO E DATI REALI
10   if (ticks mod trade-interval = 0) [
11
12     if ticks > 0 [
13       aggiorna-dati
14     ]
15
16     aggiorna-prezzo-modello
17   ]
18
19   ;; 2. AGGIORNAMENTO STATO E MOVIMENTO FISICO
20   set current-hype calcola-hype
21
22   ask turtles [
23     flock
24     repel-different-breeds
25     swim
26   ]
27
28   ;; 3. TRADING, CORRELAZIONI E PLOT
29   if (ticks mod trade-interval = 0) [
30     ask turtles [
31       compravendita-bitcoin
32     ]
33
34     aggiorna-storico-volumi
35     aggiorna-correlazione-prezzi
36
37     aggiorna-plot-prezzi-e-volume
38     aggiorna-plot-correlazione-volumi
39     aggiorna-plot-correlazione-prezzi
40   ]
41
42   aggiorna-metriche-stabilita
43   aggiorna-plot-parametri
44
45   tick
46
47 end
```

Listing 7: Procedura principale go

La procedura `go` rappresenta il ciclo dinamico principale del modello. Ogni sua esecuzione corrisponde a un passo temporale della simulazione, ma non tutte le componenti del modello vengono aggiornate con la stessa frequenza. Il movimento degli agenti avviene a ogni tick, mentre il mercato, il trading e alcuni grafici vengono aggiornati solo ogni `trade-interval` iterazioni.

All'inizio della procedura viene azzerata la variabile

`model-trading-volume`.

Questa variabile misura il volume di scambio generato dagli agenti nel tick corrente. Azzerarla all'inizio di ogni ciclo permette di registrare soltanto le operazioni avvenute nel nuovo passo temporale, evitando che il volume si accumuli indefinitamente nel tempo.

La prima parte della pipeline riguarda l'aggiornamento dei dati di mercato:

```
if (ticks mod trade-interval = 0) [ ... ].
```

La condizione

```
ticks mod trade-interval = 0
```

verifica se il numero corrente di tick è multiplo di `trade-interval`. In questo modo, il modello separa la dinamica continua del movimento dalla dinamica discreta del mercato: gli agenti possono muoversi molte volte prima che avvenga un nuovo aggiornamento di prezzo e trading.

All'interno di questo blocco, la procedura `aggiorna-dati` viene chiamata solo se `ticks > 0`. Questa scelta evita di aggiornare immediatamente i dati reali al tick iniziale, dato che il primo prezzo è già stato inizializzato nella procedura `setup`. Successivamente viene chiamata

```
aggiorna-prezzo-modello,
```

che aggiorna il prezzo endogeno generato dal modello. Il prezzo endogeno è il prezzo prodotto internamente dalla simulazione, a partire dal comportamento degli agenti, e si distingue dal prezzo reale di Bitcoin, che proviene invece dalla serie storica esterna.

La seconda parte della procedura aggiorna lo stato comportamentale e il movimento fisico degli agenti. La riga

```
set current-hype calcola-hype
```

calcola il livello corrente di hype o stress di mercato. Questa grandezza dipende dal rendimento e dalla volatilità del prezzo endogeno. Dal punto di vista economico, un mercato con alto hype è un mercato più instabile, in cui gli agenti possono diventare più reattivi e meno conservativi nelle proprie scelte.

Successivamente, tutti gli agenti eseguono tre procedure:

```
flock,      repel-different-breeds,      swim.
```

La procedura `flock` aggiorna il comportamento di allineamento collettivo, facendo sì che gli agenti tendano a coordinarsi con i propri vicini o follower. La procedura `repel-different-breeds` introduce invece una repulsione tra agenti appartenenti a specie diverse, impedendo una sovrapposizione eccessiva tra gruppi distinti. Infine, `swim` aggiorna il movimento effettivo degli agenti nello spazio.

La terza parte della pipeline riguarda trading, correlazioni e grafici. Anche questa sezione viene eseguita solo ogni `trade-interval` tick. Gli agenti eseguono la procedura

```
compravendita-bitcoin,
```

che determina se comprare o vendere Bitcoin in base alla posizione nello spazio, alla probabilità di scambio e ai vincoli di portafoglio. Gli agenti non possono comprare se non hanno liquidità sufficiente e non possono vendere Bitcoin che non possiedono.

Dopo le operazioni di trading, il modello aggiorna lo storico dei volumi tramite

```
aggiorna-storico-volumi
```

e la correlazione tra prezzi reali e prezzi simulati tramite

```
aggiorna-correlazione-prezzi.
```

La correlazione di Pearson misura quanto due serie temporali si muovono in modo linearmente simile. In questo contesto viene usata per confrontare la dinamica del modello con quella del mercato reale.

Le procedure

```
aggiorna-plot-prezzi-e-volume,      aggiorna-plot-correlazione-volumi,      aggiorna-plot-correlazione-prezzi.
```

aggiornano i grafici relativi all'andamento del prezzo, dei volumi e delle correlazioni. Poiché questi grafici dipendono da quantità di mercato, vengono aggiornati solo quando avviene anche l'aggiornamento del trading. Al di fuori del blocco temporizzato, il modello aggiorna invece le metriche di stabilità e i parametri collettivi:

```
aggiorna-metriche-stabilita,      aggiorna-plot-parametri.
```

Queste procedure vengono eseguite a ogni tick, perché dipendono dalla dinamica continua degli agenti e non soltanto dagli eventi di mercato. Possono quindi monitorare in modo più fine l'evoluzione del flocking, dell'ordine collettivo, dell'eterogeneità e dell'instabilità.

Infine, il comando

tick

fa avanzare il tempo della simulazione di una unità. La struttura complessiva della procedura può quindi essere riassunta come

mercato periodico → hype e movimento → trading periodico → metriche e plot → avanzamento temporale.

Questa organizzazione consente di rappresentare un sistema in cui gli agenti si muovono continuamente, mentre le decisioni di mercato e gli aggiornamenti dei prezzi avvengono con una frequenza più bassa e controllata.

0.8 Stima del prezzo del modello: fase reale e fase endogena

```
1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; FUNZIONE DI STIMA DEL PREZZO MODELLO
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to aggiorna-prezzo-modello
6
7      ifelse (not current-disabled?) [
8
9          ;; FASE 1: COPIA REALE
10         set model-price btc-price
11         set model-return btc-return
12         set model-volatility btc-volatility
13
14         print (word "[FASE 1 - PREZZO REALE] Giorno: " day-index " | Correlazione Media
15                   Volumi: " precision mean-volume-correlation 2)
16     ] [
17
18         ;; FASE 2: PREZZO ENDOGENO
19         if count turtles > 0 [
20             let mean-y mean [ycor] of turtles
21             let max-y max-pycor
22
23             let market-pressure (mean-y / max-y)
24
25             let prev-m-price model-price
26             set model-price max (list 1 (model-price * (1 + (price-sensitivity * market-
27               pressure))))
28
29             ifelse prev-m-price > 0 [
30                 set model-return (model-price - prev-m-price) / prev-m-price
31                 set model-volatility max (list 0.005 (abs model-return))
32             ] [
33                 set model-return 0
34                 set model-volatility 0.005
35             ]
36         ]
37
38         print (word ">>> [FASE 2 - PREZZO ENDOGENO] Giorno: " day-index " | Prezzo
39               Stimato: " precision model-price 2)
40     ]
41
42     ;; Tracciamento continuo dell'ATH sia in Fase 1 che in Fase 2
43     if model-price > btc-ath-price [
44         set btc-ath-price model-price
45     ]
46 end
```

Listing 8: Procedura di aggiornamento del prezzo del modello

La procedura **aggiorna-prezzo-modello** stabilisce come viene determinato il prezzo utilizzato internamente dalla simulazione. Il codice distingue due regimi dinamici: una prima fase in cui il modello segue direttamente il mercato reale e una seconda fase in cui il prezzo viene prodotto endogenamente dagli agenti. La condizione

```
ifelse (not current-disabled?) [ ... ] [ ... ]
```

seleziona la fase attiva. Se **current-disabled?** è falso, la corrente esterna non è disattivata e il modello si trova nella Fase 1. In questa fase il prezzo del modello viene copiato dal prezzo reale di Bitcoin:

$$P_m(t) = P_{BTC}(t).$$

Nel codice questo corrisponde a

```
set model-price btc-price.
```

Allo stesso modo, anche rendimento e volatilità del modello vengono posti uguali a quelli reali:

```
model-return = btc-return,    model-volatility = btc-volatility.
```

Questa fase può essere interpretata come una fase di calibrazione o avvio, in cui il modello resta agganciato al mercato empirico. Dal punto di vista finanziario, il rendimento misura la variazione relativa del prezzo tra due istanti consecutivi:

$$r(t) = \frac{P(t) - P(t-1)}{P(t-1)}.$$

La volatilità misura invece l'intensità delle oscillazioni del prezzo. Nel modello viene usata come grandezza in grado di influenzare la dinamica degli agenti.

Quando **current-disabled?** diventa vero, il modello entra nella Fase 2. In questa fase il prezzo non viene più copiato dal dato reale, ma viene stimato a partire dallo stato collettivo degli agenti. Il codice calcola innanzitutto la media delle coordinate verticali:

```
mean-y = mean [ycor] of turtles.
```

Nel modello, la coordinata y ha un significato comportamentale: valori positivi sono associati a una maggiore propensione all'acquisto, mentre valori negativi sono associati a una maggiore propensione alla vendita. La media $\bar{y}$ fornisce quindi una misura aggregata del sentiment della popolazione.

La pressione di mercato viene definita come

```
market-pressure = mean-y / max-y.
```

In forma matematica:

$$M(t) = \frac{\bar{y}(t)}{y_{\max}}.$$

Se $M(t) > 0$, gli agenti sono mediamente nella regione di acquisto e il modello produce una pressione rialzista sul prezzo. Se $M(t) < 0$, gli agenti sono mediamente nella regione di vendita e il modello produce una pressione ribassista.

Il prezzo endogeno viene aggiornato secondo la regola

$$P_m(t+1) = P_m(t) [1 + \alpha M(t)],$$

dove α è rappresentato da **price-sensitivity**. Nel codice:

```
model-price * (1 + (price-sensitivity * market-pressure)).
```

La funzione

```
max (list 1 ...)
```

impone un limite inferiore pari a 1, evitando che il prezzo stimato diventi zero o negativo. Questa scelta è coerente con l'interpretazione economica del prezzo: il prezzo di un'attività finanziaria non dovrebbe assumere valori negativi.

Dopo l'aggiornamento del prezzo, il codice calcola il rendimento endogeno:

$$r_m(t) = \frac{P_m(t) - P_m(t-1)}{P_m(t-1)}.$$

Nel codice:

```
set model-return (model-price - prev-m-price) / prev-m-price.
```

La volatilità endogena viene quindi posta uguale al valore assoluto del rendimento, ma con una soglia minima:

$$\sigma_m(t) = \max(0.005, |r_m(t)|).$$

La soglia 0.005 rappresenta una volatilità di fondo. Essa evita che la volatilità diventi esattamente nulla e impedisce che eventuali meccanismi dipendenti dalla volatilità, come la corrente del modello, si spengano completamente.

Infine, la procedura aggiorna il massimo prezzo raggiunto:

```
if model-price > btc-ath-price [ set btc-ath-price model-price ].
```

La variabile `btc-ath-price` conserva quindi il massimo valore osservato del prezzo usato dal modello, sia durante la fase agganciata al prezzo reale sia durante la fase endogena. La sigla ATH indica *All-Time High*, cioè il massimo storico o massimo raggiunto nella finestra di simulazione considerata.

Questa procedura è centrale perché realizza il passaggio da un modello guidato dai dati reali a un modello autonomo. Nella prima fase il mercato reale fornisce il riferimento principale; nella seconda fase, invece, il prezzo emerge dallo stato collettivo degli agenti e dalla loro distribuzione nello spazio comportamentale.

0.9 Corrente dinamica nelle due fasi del modello

```

1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; CORRENTE -- COMPONENTE X (DINAMICA FASE 1 / FASE 2)
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to-report my-current-x
6    let active-vol (ifelse-value (not current-disabled?) [ btc-volatility ] [ model-
7      volatility ])
8    report k * active-vol
9  end
10
11 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
12 ;; CORRENTE -- COMPONENTE Y (DINAMICA FASE 1 / FASE 2)
13 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
14
15 to-report my-current-y
16   let active-ret (ifelse-value (not current-disabled?) [ btc-return ] [ model-
17     return ])
18   ifelse breed = agents-a [
19     report h * active-ret
20   ] [
21     ifelse breed = agents-b [
22       report (- h * active-ret)
23     ] [
24       report 0
25     ]
26   ]
27 end

```

Listing 9: Componenti della corrente nelle fasi reale ed endogena

Queste due procedure definiscono la corrente esterna che agisce sugli agenti. La corrente è divisa in due componenti:

$$J_x, \quad J_y.$$

La componente orizzontale è calcolata dal reporter `my-current-x`, mentre la componente verticale è calcolata da `my-current-y`. Entrambe le componenti dipendono dalla fase dinamica in cui si trova il modello.

La riga

```
let active-vol (ifelse-value (not current-disabled?) [ btc-volatility ] [ model-volatility ])
```

seleziona quale volatilità usare. Se `current-disabled?` è falso, il modello si trova nella Fase 1 e usa la volatilità reale di Bitcoin, `btc-volatility`. Se invece `current-disabled?` è vero, il modello si trova nella Fase 2 e usa la volatilità endogena, `model-volatility`.

La componente orizzontale della corrente è quindi

$$J_x(t) = k \sigma(t),$$

dove k è il parametro di intensità della corrente e $\sigma(t)$ è la volatilità attiva, reale nella Fase 1 ed endogena nella Fase 2.

Dal punto di vista economico, la volatilità misura l'intensità delle variazioni di prezzo. Una volatilità elevata descrive un mercato più instabile o turbolento. Nel modello, questa instabilità viene tradotta in una spinta orizzontale più forte sugli agenti. Poiché la coordinata x è collegata alla probabilità di trading, la corrente orizzontale può essere interpretata come un aumento della reattività degli agenti durante fasi di mercato più intense.

Il reporter `my-current-y` definisce invece la componente verticale della corrente. In questo caso la grandezza rilevante non è la volatilità, ma il rendimento:

$$r(t) = \frac{P(t) - P(t-1)}{P(t-1)}.$$

La riga

```
let active-ret (ifelse-value (not current-disabled?) [ btc-return ] [ model-return ])
```

sceglie il rendimento reale nella Fase 1 e il rendimento endogeno nella Fase 2.

La componente verticale dipende anche dalla specie dell'agente. Per gli agenti della specie `agents-a`, il codice restituisce

$$J_y^{(A)}(t) = h r(t),$$

mentre per gli agenti della specie `agents-b` restituisce

$$J_y^{(B)}(t) = -h r(t).$$

Per gli agenti della specie `agents-c`, invece, la componente verticale è nulla:

$$J_y^{(C)}(t) = 0.$$

Il parametro h controlla l'intensità con cui il rendimento influenza il moto verticale degli agenti. Poiché nel modello la coordinata y distingue le zone comportamentali di acquisto e vendita, questa scelta assegna alle specie A e B una risposta opposta allo stesso segnale di mercato.

Se il rendimento è positivo, gli agenti A vengono spinti verso l'alto, cioè verso la regione associata all'acquisto, mentre gli agenti B vengono spinti verso il basso, cioè verso la regione associata alla vendita. Se il rendimento è negativo, l'effetto si inverte. La specie C, invece, non riceve alcuna spinta verticale diretta dal rendimento e può essere interpretata come una popolazione più neutra rispetto alla direzione del mercato.

Questa struttura consente di rappresentare tre categorie comportamentali differenti: una specie che segue il segnale di rendimento, una specie che reagisce in modo opposto e una specie neutrale. Il passaggio dalla Fase 1 alla Fase 2 mantiene la stessa forma della corrente, ma cambia la sorgente del segnale: inizialmente il mercato reale guida la dinamica, successivamente il segnale viene prodotto dal mercato endogeno generato dagli agenti.

0.10 Movimento degli agenti con corrente, riflessione ai bordi e rumore

```
1  ;;
2  ;; MOVIMENTO
3  ;;
4
5  to swim
6
7      set velocity-x velocity-x + my-current-x
8      set velocity-y velocity-y + my-current-y
9
10     let next-x xcor + velocity-x
11     let next-y ycor + velocity-y
12
13     if next-x > max-pxcor or next-x < min-pxcor [
14         set velocity-x (- velocity-x)
15         set next-x max (list min-pxcor (min (list max-pxcor next-x)))
16     ]
17
18     if next-y > max-pycor or next-y < min-pycor [
19         set velocity-y (- velocity-y)
20         set next-y max (list min-pycor (min (list max-pycor next-y)))
21     ]
22
23     setxy next-x next-y
24
25     let random-angle random-float (2 * temperature) - temperature
26     let old-vx velocity-x
27     let old-vy velocity-y
28
29     set velocity-x (old-vx * cos random-angle) - (old-vy * sin random-angle)
30     set velocity-y (old-vx * sin random-angle) + (old-vy * cos random-angle)
31
32     if velocity-x != 0 or velocity-y != 0 [
33         set heading atan velocity-x velocity-y
34     ]
35
36 end
```

Listing 10: Procedura di movimento degli agenti

La procedura `swim` aggiorna il movimento degli agenti nello spazio. A differenza di una dinamica basata soltanto su `heading` e `fd`, qui ogni agente possiede una velocità vettoriale, descritta dalle due componenti

$$v_x = \text{velocity-x}, \quad v_y = \text{velocity-y}.$$

La prima parte della procedura aggiunge alla velocità dell'agente la corrente esterna del modello:

```
set velocity-x velocity-x + my-current-x,

set velocity-y velocity-y + my-current-y.
```

La corrente rappresenta l'effetto del mercato sul moto degli agenti. La componente orizzontale dipende dalla volatilità, mentre la componente verticale dipende dal rendimento e dalla specie dell'agente. In questo modo, l'andamento del mercato influenza direttamente la traiettoria degli agenti nello spazio comportamentale. Dopo l'aggiornamento della velocità, il codice calcola la posizione prevista al passo successivo:

$$x_{\text{next}} = x + v_x, \quad y_{\text{next}} = y + v_y.$$

Nel codice:

```
let next-x xcor + velocity-x,    let next-y ycor + velocity-y.
```

La parte successiva gestisce le pareti riflettenti. Se la posizione prevista supera il bordo destro o sinistro del mondo, la componente orizzontale della velocità viene invertita:

$$v_x \longrightarrow -v_x.$$

Questo è implementato da

```
set velocity-x (- velocity-x).
```

Successivamente, la posizione viene riportata all'interno dei limiti del mondo tramite

```
max (list min-pxcor (min (list max-pxcor next-x))).
```

Questa istruzione forza **next-x** a rimanere compreso tra **min-pxcor** e **max-pxcor**. Lo stesso procedimento viene applicato alla componente verticale: se l'agente supera il bordo superiore o inferiore, la velocità verticale viene invertita,

$$v_y \longrightarrow -v_y,$$

e la coordinata **next-y** viene riportata dentro i limiti ammessi.

Il comando

```
setxy next-x next-y
```

aggiorna quindi la posizione effettiva dell'agente.

La seconda parte della procedura introduce una perturbazione casuale della direzione di moto. Viene generato un angolo casuale:

$$\text{random-angle} \in [-\text{temperature}, \text{temperature}).$$

Il parametro **temperature** controlla quindi l'intensità del rumore angolare. Valori bassi producono traiettorie più regolari; valori elevati rendono il moto più disordinato.

La velocità viene poi ruotata usando la matrice di rotazione bidimensionale:

$$\begin{pmatrix} v'_x \\ v'_y \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} v_x \\ v_y \end{pmatrix}.$$

Nel codice, questa rotazione è implementata dalle righe

```
set velocity-x (old-vx * cos random-angle) - (old-vy * sin random-angle)
```

e

```
set velocity-y (old-vx * sin random-angle) + (old-vy * cos random-angle).
```

Infine, se la velocità non è nulla, il valore di **heading** viene aggiornato in modo coerente con la direzione del vettore velocità:

```
set heading atan velocity-x velocity-y.
```

In questo modo, la rappresentazione grafica dell'agente rimane allineata alla direzione effettiva del suo moto. Dal punto di vista modellistico, questa procedura combina tre effetti: la corrente generata dal mercato, la riflessione ai bordi del dominio e il rumore casuale. Il risultato è una dinamica in cui gli agenti sono spinti dal segnale finanziario, rimangono confinati nello spazio e mantengono una componente stocastica che può ostacolare o modificare l'emergere del flocking.

0.11 Probabilità di compravendita

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; PROBABILITA' DI COMPRACENDITA
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to-report probabilita-scambio
6
7   let ampiezza-x (max-pxcor - min-pxcor)
8
9   if ampiezza-x = 0 [ report 0 ]
10
```

```

11  let p ((xcor - min-pxcor) / ampiezza-x)
12
13  report min (list 1 (max (list 0 p)))
14
15  end

```

Listing 11: Reporter per la probabilità di scambio

Il reporter `probabilita-scambio` assegna a ogni agente una probabilità di effettuare una compravendita in base alla sua posizione orizzontale nel mondo NetLogo.

La variabile

$$\text{ampiezza-x} = \text{max-pxcor} - \text{min-pxcor}$$

rappresenta la larghezza totale del dominio lungo l'asse x . Se tale ampiezza è nulla, il reporter restituisce direttamente 0, evitando una divisione per zero.

La probabilità grezza viene poi calcolata come

$$p = \frac{x - x_{\min}}{x_{\max} - x_{\min}}.$$

Nel codice:

```
let p ((xcor - min-pxcor) / ampiezza-x).
```

In questo modo, un agente vicino al bordo sinistro del mondo ha una probabilità di scambio prossima a 0, mentre un agente vicino al bordo destro ha una probabilità prossima a 1. La coordinata x viene quindi interpretata come una misura della propensione operativa dell'agente: più l'agente si trova verso destra, maggiore è la probabilità che partecipi al mercato.

L'ultima riga,

```
report min (list 1 (max (list 0 p))),
```

serve a vincolare il valore finale nell'intervallo

$$0 \leq p \leq 1.$$

Questa operazione evita che eventuali valori numerici anomali producano probabilità negative o maggiori di uno. Il reporter restituisce quindi sempre un valore valido interpretabile come probabilità di compravendita.

0.12 Compravendita di Bitcoin

```

1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; COMPRAVENDITA DI BITCOIN
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to compravendita-bitcoin
6
7    let current-p model-price
8
9    if current-p > 0 [
10
11      let p probabilita-scambio
12
13      let quantita-acquisto min (list trade-size (floor (cash-balance / current-p)))
14      let quantita-vendita min (list trade-size bitcoin-held)
15
16      if ycor >= 0 [
17        if (random-float 1 < p) and (quantita-acquisto > 0) [
18          let costo quantita-acquisto * current-p
19          set cash-balance cash-balance - costo
20          set bitcoin-held bitcoin-held + quantita-acquisto
21
22          set model-trading-volume model-trading-volume + quantita-acquisto
23        ]
24      ]

```

```

25
26   if ycor < 0 [
27       if (random-float 1 < p) and (quantita-vendita > 0) [
28           let ricavo quantita-vendita * current-p
29           set bitcoin-held bitcoin-held - quantita-vendita
30           set cash-balance cash-balance + ricavo
31
32           set model-trading-volume model-trading-volume + quantita-vendita
33       ]
34   ]
35 ]
36
37 end

```

Listing 12: Procedura di compravendita di Bitcoin degli agenti

La procedura `compravendita-bitcoin` descrive il comportamento di trading degli agenti. Ogni agente può comprare o vendere Bitcoin usando il prezzo endogeno del modello, memorizzato nella variabile

`current-p = model-price.`

Il blocco principale viene eseguito solo se il prezzo è positivo:

`if current-p > 0 [ ... ].`

Questa condizione evita operazioni economicamente non valide, come acquisti o vendite a prezzo nullo o negativo.

La probabilità che l'agente effettui uno scambio è data da

`p = probabilita-scambio.`

Questa probabilità dipende dalla posizione orizzontale dell'agente: più l'agente si trova verso destra, maggiore è la probabilità che partecipi al mercato.

La quantità massima acquistabile è calcolata come

$$\text{quantita-acquisto} = \min \left(\text{trade-size}, \left\lfloor \frac{\text{cash-balance}}{\text{current-p}} \right\rfloor \right).$$

Questa formula impedisce all'agente di comprare più Bitcoin di quanto possa permettersi con la liquidità disponibile. La funzione `floor` arrotonda per difetto il numero di Bitcoin acquistabili.

La quantità massima vendibile è invece

`quantita-vendita = min(trade-size, bitcoin-held).`

In questo modo l'agente non può vendere più Bitcoin di quelli effettivamente posseduti.

La coordinata verticale `ycor` stabilisce il tipo di operazione. Se

`ycor >= 0,`

l'agente si trova nella regione associata all'acquisto. In questo caso, se un numero casuale tra 0 e 1 è minore della probabilità `p` e se la quantità acquistabile è positiva, l'agente compra Bitcoin. Il costo dell'operazione è

`costo = quantita-acquisto * current-p.`

Il saldo liquido diminuisce di tale costo, mentre la quantità di Bitcoin detenuta aumenta della quantità acquistata.

Se invece

`ycor < 0,`

l'agente si trova nella regione associata alla vendita. Se la vendita viene estratta casualmente e l'agente possiede Bitcoin sufficienti, il ricavo è

`ricavo = quantita-vendita * current-p.`

In questo caso la quantità di Bitcoin detenuta diminuisce, mentre la liquidità aumenta del ricavo ottenuto. Dopo ogni acquisto o vendita riuscita, il volume di trading generato dal modello viene aggiornato:

`model-trading-volume ← model-trading-volume + quantita.`

Questa variabile misura il numero complessivo di Bitcoin scambiati dagli agenti nel tick corrente. La procedura collega quindi la posizione spaziale degli agenti alla loro attività di mercato: la coordinata y determina la direzione dell'operazione, mentre la coordinata x ne influenza la probabilità.

0.13 Fattore di isteresi dei volumi basato sul prezzo

```

1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; FATTORE DI SCALING VOLUMI (UNIFICATO BASATO SU PREZZO)
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to-report volume-hysteresis-factor
6
7      let initial-p max (list 1 btc-initial-price)
8      let current-p max (list 1 model-price)
9      let ath-p      max (list current-p btc-ath-price)
10
11     let ratio-current (current-p / initial-p)
12     let ratio-ath      (ath-p / initial-p)
13
14     ;; Formula continua dell'isteresi valida sia in Fase 1 che in Fase 2
15     let price-hysteresis-state ratio-current + (beta-vol * (ratio-ath - ratio-current
16       ))
17
18     report max (list 0.01 price-hysteresis-state)
19 end

```

Listing 13: Reporter per il fattore di scaling dei volumi

Il reporter `volume-hysteresis-factor` calcola un fattore di scaling dei volumi a partire dall'andamento del prezzo. L'idea è che il volume di mercato non dipenda soltanto dal prezzo corrente, ma anche dalla memoria del massimo prezzo raggiunto in passato.

Le prime tre variabili definiscono prezzi sempre positivi:

$$\text{initial-p} = \max(1, \text{btc-initial-price}),$$

$$\text{current-p} = \max(1, \text{model-price}),$$

$$\text{ath-p} = \max(\text{current-p}, \text{btc-ath-price}).$$

L'uso di $\max(1, \cdot)$ evita divisioni per zero o valori troppo piccoli. La variabile `ath-p` rappresenta invece il massimo prezzo rilevante per il modello, cioè il maggiore tra il prezzo corrente e il massimo storico registrato. Il codice calcola poi due rapporti normalizzati rispetto al prezzo iniziale:

$$\text{ratio-current} = \frac{\text{current-p}}{\text{initial-p}},$$

$$\text{ratio-ath} = \frac{\text{ath-p}}{\text{initial-p}}.$$

Il primo rapporto misura quanto il prezzo corrente è cresciuto rispetto al valore iniziale. Il secondo misura invece quanto il massimo storico è cresciuto rispetto al valore iniziale.

Il fattore di isteresi viene definito come

$$H_V = \text{ratio-current} + \beta_{\text{vol}} (\text{ratio-ath} - \text{ratio-current}).$$

Nel codice:

$$\text{price-hysteresis-state} = \text{ratio-current} + (\text{beta-vol} * (\text{ratio-ath} - \text{ratio-current})).$$

Il parametro `beta-vol` controlla quanto peso viene dato alla memoria del massimo storico. Se `beta-vol` è vicino a 0, il fattore dipende quasi solo dal prezzo corrente. Se invece è più alto, il fattore conserva maggiormente l'effetto dei picchi di prezzo passati.

In questo senso si parla di isteresi: lo stato attuale del volume non dipende solo dalla situazione presente, ma anche dalla storia precedente del prezzo. Anche se il prezzo scende dopo aver raggiunto un massimo, il modello può mantenere un livello di volume più elevato a causa della memoria del picco.

L'ultima riga

```
report max (list 0.01 price-hysteresis-state)
```

impone un limite inferiore pari a 0.01. In questo modo il fattore di scaling dei volumi resta sempre positivo e non può annullarsi completamente.

0.14 Volume del modello espresso in dollari

```
1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; VOLUME DEL MODELLO IN DOLLARI
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to-report model-trading-volume-usd
6
7      let current-p model-price
8      if current-p <= 0 [ report 0 ]
9
10     report (model-trading-volume * current-p) * volume-hysteresis-factor
11
12 end
```

Listing 14: Reporter per il volume di trading del modello in dollari

Il reporter `model-trading-volume-usd` converte il volume di trading generato dagli agenti in un controvalore espresso in dollari.

La variabile locale

```
current-p = model-price
```

memorizza il prezzo endogeno corrente del modello. Questo prezzo viene usato per trasformare il numero di Bitcoin scambiati in un valore monetario.

Il controllo

```
if current-p <= 0 [ report 0 ]
```

serve a evitare che il volume venga calcolato usando un prezzo nullo o negativo. In tal caso il reporter restituisce direttamente 0, perché non avrebbe senso economico calcolare un controvalore di mercato con un prezzo non positivo.

Se il prezzo è valido, il volume in dollari viene calcolato come

$$V_{\text{model}}^{USD} = V_{\text{model}}^{BTC} \cdot P_m \cdot H_V,$$

dove V_{model}^{BTC} è il volume di Bitcoin scambiato dagli agenti, P_m è il prezzo endogeno del modello e H_V è il fattore di isteresi dei volumi.

Nel codice questa formula corrisponde a

```
(model-trading-volume * current-p) * volume-hysteresis-factor.
```

Il termine `model-trading-volume * current-p` trasforma il volume da Bitcoin a dollari. Il fattore `volume-hysteresis-factor` introduce invece una correzione legata alla memoria del prezzo: i volumi non dipendono soltanto dagli scambi correnti, ma anche dalla dinamica storica del prezzo e dal massimo raggiunto in precedenza.

In questo modo il modello produce un volume simulato in dollari confrontabile con il volume reale di mercato.

0.15 Lettura dei dati storici da file CSV

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; LETTURA FILE CSV
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to carica-dati-da-csv [ nome-file ]
6   set prices []
7   file-close-all
8
9   carefully [
10     let dati csv:from-file nome-file
11
12     if length dati > 0 [
13       let prima-riga item 0 dati
14       if not is-number? item 1 prima-riga [
15         set dati but-first dati
16       ]
17     ]
18
19     let t 0
20     foreach dati [ riga ->
21       if length riga >= 2 [
22         let p item 1 riga
23         if is-number? p [
24           let vol 0
25           if length riga >= 3 [
26             if is-number? item 2 riga [
27               set vol item 2 riga
28             ]
29           ]
30           set prices lput (list t p vol) prices
31           set t t + 1
32         ]
33       ]
34     ]
35
36     print (word "CSV caricato con successo! Estratti " length prices " record
37               storici validi.")
38   ] [
39     file-close-all
40     user-message (word "Errore nella lettura del file CSV: " nome-file "\nVerifica
41                     che il file sia nella stessa cartella.")
42   ]
43 end
```

Listing 15: Procedura per il caricamento dei dati da un file CSV

La procedura `carica-dati-da-csv` permette di caricare una serie storica da un file CSV esterno. L'argomento `nome-file` indica il nome del file da leggere, che deve trovarsi nella cartella del modello NetLogo. All'inizio la lista `prices` viene svuotata:

```
set prices [].
```

In questo modo, ogni nuovo caricamento sostituisce i dati precedenti invece di aggiungersi a essi. Il comando `file-close-all` chiude eventuali file rimasti aperti da letture precedenti.

La lettura vera e propria avviene dentro un blocco `carefully`. Questo serve a intercettare eventuali errori, ad esempio se il file non esiste, se il nome è scritto male oppure se il file non è nella cartella corretta.

Il comando

```
let dati csv:from-file nome-file
```

legge il file CSV e lo trasforma in una lista di righe. Ogni riga è a sua volta una lista contenente i valori delle colonne.

Il blocco successivo controlla se la prima riga è un'intestazione. Se nella seconda colonna della prima riga non è presente un numero, allora il codice interpreta quella riga come header e la rimuove:

```
set dati but-first dati.
```

Successivamente viene introdotta la variabile `t`, che rappresenta l'indice temporale dei dati caricati. Per ogni riga del file, il codice controlla che siano presenti almeno due colonne. La seconda colonna viene interpretata come prezzo:

```
let p item 1 riga.
```

Se `p` è un numero, allora la riga viene considerata valida. Il volume viene inizializzato a zero:

```
let vol 0.
```

Se esiste anche una terza colonna e questa contiene un valore numerico, allora quel valore viene usato come volume:

```
set vol item 2 riga.
```

Ogni record valido viene infine inserito nella lista `prices` nella forma

$$(t, p, vol),$$

cioè:

```
list t p vol.
```

Nel modello, quindi, ogni elemento di `prices` contiene tre informazioni: l'indice temporale, il prezzo e il volume associato. Dopo ogni riga valida, `t` viene incrementato di uno, così da costruire una sequenza temporale ordinata.

Se la lettura va a buon fine, il modello stampa un messaggio con il numero di record storici validi estratti. Se invece si verifica un errore, viene chiuso ogni file aperto e compare un messaggio per avvisare l'utente che il CSV non è stato letto correttamente.

0.16 Gestione degli scenari di simulazione

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; GESTIONE SCENARI
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to scarica-dati
6
7   if scenario = "csv" [
8     carica-dati-da-csv "dati_btc.csv"
9     stop
10  ]
11
12   if scenario = "api-live" [
13     set prices genera-prezzi-api
14     stop
15  ]
16
17   if scenario = "crash" [
18     set prices genera-crash 60000
19     stop
20  ]
21
22   if scenario = "bull-market" [
23     set prices genera-prezzi 30000 0.015 0.01
24     stop
25  ]
26
27   if scenario = "bear-market" [
28     set prices genera-prezzi 60000 -0.015 0.01
```

```

29     stop
30 ]
31
32 if scenario = "sideways" [
33     set prices genera-prezzi 40000 0 0.005
34     stop
35 ]
36
37 if scenario = "bubble" [
38     set prices genera-bolla 30000
39     stop
40 ]
41
42 end

```

Listing 16: Procedura per la selezione dello scenario di mercato

La procedura `scarica-dati` seleziona quale serie storica utilizzare per inizializzare il modello. La scelta dipende dal valore della variabile `scenario`, che può corrispondere a diversi tipi di mercato. Se lo scenario è

`"csv",`

il modello carica i dati dal file esterno `dati_btc.csv` tramite la procedura

`carica-dati-da-csv "dati_btc.csv".`

In questo caso la simulazione usa una serie storica salvata localmente. Se lo scenario è

`"api-live",`

la lista `prices` viene costruita tramite

`set prices genera-prezzi-api.`

Questo scenario permette di usare dati ottenuti da una sorgente esterna, invece di dati generati artificialmente. Gli altri scenari producono invece serie sintetiche. Lo scenario

`"crash"`

genera un andamento caratterizzato da una forte caduta del prezzo, partendo dal valore iniziale 60000. Lo scenario

`"bull-market"`

genera una dinamica crescente, con prezzo iniziale 30000, drift positivo 0.015 e rumore 0.01. Lo scenario

`"bear-market"`

usa invece un drift negativo pari a -0.015 , simulando un mercato tendenzialmente ribassista. Lo scenario

`"sideways"`

rappresenta un mercato laterale, cioè privo di una direzione dominante. Infatti il drift è nullo:

`genera-prezzi 40000 0 0.005.`

Infine, lo scenario

`"bubble"`

genera una dinamica di bolla, partendo da un prezzo iniziale pari a 30000. Dopo ogni scelta viene eseguito il comando

`stop.`

Questo interrompe la procedura appena uno scenario è stato riconosciuto, evitando che vengano controllati inutilmente anche gli scenari successivi.

La procedura permette quindi di usare lo stesso modello in condizioni di mercato diverse: dati reali da CSV, dati esterni tramite API oppure scenari sintetici come crescita, crisi, mercato laterale e bolla speculativa.

0.17 Generazione dei prezzi tramite API live

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; PREZZO LIVE DA API
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to-report genera-prezzi-api
6
7   let url "https://api.coinbase.com/v2/prices/BTC-USD/spot"
8   let raw-json ""
9   let fetch-failed? false
10
11   carefully [
12     set raw-json fetch:url url
13   ] [
14     set fetch-failed? true
15   ]
16
17   if fetch-failed? or raw-json = "" [
18     user-message "Errore richiesta API\nUso del trend sintetico di fallback."
19     report genera-prezzi 40000 0 0.005
20   ]
21
22   let data table:from-json raw-json
23   let data-map table:get data "data"
24   let current-price read-from-string (table:get data-map "amount")
25
26   let result []
27   let p 40000
28   let t 0
29
30   while [t < (num-days - 1)] [
31     let r (random-normal 0 0.005)
32     set p p * (1 + r)
33     set result lput (list t p 0) result
34     set t t + 1
35   ]
36
37   set result lput (list t current-price 0) result
38
39   report result
40
41 end
```

Listing 17: Reporter per la costruzione di una serie prezzi con prezzo finale live da API

Il reporter `genera-prezzi-api` costruisce una serie di prezzi usando un prezzo live ottenuto tramite API. L'obiettivo non è scaricare un'intera serie storica reale, ma generare una traiettoria sintetica che termina con il prezzo corrente di Bitcoin.

La variabile

`url`

contiene l'indirizzo dell'API da cui viene richiesto il prezzo spot di BTC-USD. Le variabili `raw-json` e `fetch-failed?` servono rispettivamente a memorizzare la risposta grezza dell'API e a registrare l'eventuale fallimento della richiesta.

La chiamata esterna viene eseguita dentro un blocco `carefully`:

```
set raw-json fetch:url url.
```

Se la richiesta fallisce, la variabile `fetch-failed?` viene posta uguale a `true`. In questo modo il modello non si blocca in caso di assenza di connessione, errore dell'API o risposta non valida. Se la richiesta non va a buon fine, oppure se la risposta è vuota, il modello mostra un messaggio all'utente e usa una serie sintetica di fallback:

```
genera-prezzi 40000 0 0.005.
```

Questo fallback rappresenta un mercato laterale con prezzo iniziale 40000, drift nullo e rumore pari a 0.005. Se invece la risposta dell'API è valida, il testo JSON viene convertito in una struttura leggibile tramite

```
table:from-json raw-json.
```

Il codice accede poi al campo `"data"` e recupera il valore `"amount"`, che contiene il prezzo corrente. Poiché tale valore arriva come stringa, viene trasformato in numero con

```
read-from-string.
```

La seconda parte del reporter costruisce la serie `result`. La variabile `p` parte da 40000, mentre `t` rappresenta l'indice temporale. Finché

$$t < \text{num-days} - 1,$$

il modello genera un rendimento casuale

$$r \sim \mathcal{N}(0, 0.005)$$

e aggiorna il prezzo secondo la regola

$$p_{t+1} = p_t(1 + r).$$

Ogni valore generato viene aggiunto alla lista `result` nella forma

$$(t, p, 0),$$

dove il terzo elemento rappresenta il volume, qui posto uguale a zero.

Alla fine del ciclo, l'ultimo elemento della serie viene sostituito dal prezzo live ottenuto tramite API:

```
set result lput (list t current-price 0) result.
```

Il reporter restituisce quindi una lista di record temporali in cui i primi valori sono generati sinteticamente, mentre l'ultimo prezzo coincide con il prezzo corrente letto dall'API.

0.18 Generazione di scenari sintetici di prezzo

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; GENERA PREZZI SINTETICI
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to-report genera-prezzi [ prezzo-iniziale drift sigma ]
6
7   let result []
8   let p prezzo-iniziale
9   let t 0
10
11   while [t < num-days] [
12     let r drift + random-normal 0 sigma
13     set p p * (1 + r)
14     if p < 1 [ set p 1 ]
15     set result lput (list t p 0) result
16     set t t + 1
17   ]
18
19   report result
20
21 end
```

```

22
23
24 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
25 ;; GENERA CRASH SINTETICO
26 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
27
28 to-report genera-crash [prezzo-iniziale]
29
30   let result []
31   let p prezzo-iniziale
32   let t 0
33
34   while [t < num-days] [
35     let r 0
36
37     ifelse t < 100 [
38       set r random-normal 0 0.005
39     ][
40       ifelse t < 110 [
41         set r -0.25 + random-normal 0 0.02
42       ][
43         set r random-normal 0.003 0.012
44       ]
45     ]
46
47     set p p * (1 + r)
48     if p < 1 [ set p 1 ]
49     set result lput (list t p 0) result
50     set t t + 1
51   ]
52
53   report result
54
55 end
56
57
58 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
59 ;; GENERA BOLLA SINTETICA
60 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
61
62 to-report genera-bolla [prezzo-iniziale]
63
64   let result []
65   let p prezzo-iniziale
66   let t 0
67
68   while [t < num-days] [
69     let r 0
70
71     ifelse t < 100 [
72       set r 0.025 + random-normal 0 0.008
73     ][
74       ifelse t < 110 [
75         set r 0.08 + random-normal 0 0.02
76       ][
77         ifelse t < 115 [
78           set r -0.3 + random-normal 0 0.025
79         ][
80           set r -0.002 + random-normal 0 0.015
81         ]
82       ]
83     ]
84
85     set p p * (1 + r)
86     if p < 1 [ set p 1 ]

```

```

87     set result lput (list t p 0) result
88     set t t + 1
89 ]
90
91     report result
92
93 end

```

Listing 18: Reporter per la generazione di prezzi sintetici, crash e bolla

Questi tre reporter generano serie temporali sintetiche di prezzo, usate quando non si vogliono caricare dati reali o quando si desidera studiare il comportamento del modello in scenari controllati.

Il reporter **genera-prezzi** produce una dinamica generica a partire da tre parametri: **prezzo-iniziale**, **drift** e **sigma**. Il prezzo parte dal valore iniziale assegnato e, a ogni passo temporale, viene aggiornato tramite un rendimento casuale:

$$r = \text{drift} + \mathcal{N}(0, \text{sigma}).$$

Il parametro **drift** rappresenta la tendenza media del mercato: se è positivo genera una crescita media, se è negativo genera una discesa media, se è nullo produce un mercato laterale. Il parametro **sigma** controlla invece l'intensità delle oscillazioni casuali.

Il prezzo viene aggiornato secondo la regola

$$p_{t+1} = p_t(1 + r).$$

La condizione

```
if p < 1 [ set p 1 ]
```

impone un limite inferiore al prezzo, evitando che la serie assuma valori nulli o negativi. Ogni osservazione viene salvata nella lista **result** nella forma

$$(t, p, 0),$$

dove t è l'indice temporale, p è il prezzo e il terzo valore rappresenta il volume, qui posto uguale a zero.

Il reporter **genera-crash** costruisce invece uno scenario di crollo del mercato. La dinamica è divisa in tre fasi. Per i primi 100 tick il prezzo oscilla intorno a un rendimento medio nullo:

$$r \sim \mathcal{N}(0, 0.005).$$

Tra il tick 100 e il tick 109, il modello introduce una forte caduta:

$$r = -0.25 + \mathcal{N}(0, 0.02).$$

Dopo il crash, il rendimento torna moderatamente positivo:

$$r \sim \mathcal{N}(0.003, 0.012).$$

Questo permette di simulare una fase inizialmente stabile, seguita da un crollo rapido e da una successiva fase di parziale recupero.

Il reporter **genera-bolla** genera invece uno scenario di bolla speculativa. Anche in questo caso la dinamica è divisa in più fasi. Nei primi 100 tick il prezzo cresce in modo sostenuto:

$$r = 0.025 + \mathcal{N}(0, 0.008).$$

Tra il tick 100 e il tick 109, la crescita accelera ulteriormente:

$$r = 0.08 + \mathcal{N}(0, 0.02).$$

Tra il tick 110 e il tick 114, la bolla esplode e il prezzo subisce una brusca caduta:

$$r = -0.3 + \mathcal{N}(0, 0.025).$$

Infine, dopo il collasso, il mercato entra in una fase discendente e più rumorosa:

$$r = -0.002 + \mathcal{N}(0, 0.015).$$

Queste tre procedure permettono quindi di confrontare il comportamento degli agenti in mercati con caratteristiche molto diverse: crescita regolare, ribasso, stabilità laterale, crash improvviso e bolla speculativa.

0.19 Aggiornamento giornaliero dei dati di mercato

```
1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; AGGIORNAMENTO GIORNALIERO
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to aggiorna-dati
6
7    if not is-list? prices [ stop ]
8    if day-index >= length prices [ stop ]
9
10   let record item day-index prices
11   let new-price item 1 record
12
13   ifelse length record >= 3 [
14     set real-trading-volume-usd item 2 record
15   ] [
16     set real-trading-volume-usd 0
17   ]
18
19   if real-volume-initial = 0 and real-trading-volume-usd > 0 [
20     set real-volume-initial real-trading-volume-usd
21     set real-volume-ath real-trading-volume-usd
22   ]
23
24   if real-trading-volume-usd > real-volume-ath [
25     set real-volume-ath real-trading-volume-usd
26   ]
27
28   ifelse btc-prev-price > 0 [
29     set btc-return (new-price - btc-prev-price) / btc-prev-price
30     set btc-volatility abs btc-return
31   ] [
32     set btc-return 0
33     set btc-volatility 0
34   ]
35
36   set btc-prev-price btc-price
37   set btc-price new-price
38
39   if btc-initial-price = 0 and btc-price > 0 [
40     set btc-initial-price btc-price
41     set btc-ath-price btc-price
42   ]
43
44   if btc-price > btc-ath-price [
45     set btc-ath-price btc-price
46   ]
47
48   set day-index day-index + 1
49
50 end
```

Listing 19: Procedura di aggiornamento giornaliero dei prezzi e dei volumi reali

La procedura `aggiorna-dati` aggiorna i dati reali di mercato letti dalla lista `prices`. Ogni elemento della lista rappresenta un record temporale contenente almeno il prezzo e, se disponibile, anche il volume reale di scambio.

All'inizio vengono effettuati due controlli di sicurezza:

```
if not is-list? prices [ stop ]
```

e

```
if day-index >= length prices [ stop ].
```

Il primo controllo verifica che `prices` sia effettivamente una lista; il secondo evita di leggere oltre la fine della serie storica disponibile.

Il record corrente viene estratto tramite

```
let record item day-index prices,
```

mentre il nuovo prezzo viene preso dalla seconda posizione del record:

```
let new-price item 1 record.
```

Se il record contiene almeno tre elementi, il terzo valore viene interpretato come volume reale in dollari:

```
set real-trading-volume-usd item 2 record.
```

Se invece il volume non è disponibile, viene posto uguale a zero. Questa scelta permette alla procedura di funzionare sia con serie storiche complete, contenenti prezzo e volume, sia con serie contenenti soltanto il prezzo.

Il blocco successivo inizializza il volume reale iniziale e il massimo volume raggiunto:

```
real-volume-initial,    real-volume-ath.
```

Il volume iniziale viene impostato solo quando viene osservato il primo volume positivo. Successivamente, se il volume reale corrente supera il massimo registrato, la variabile `real-volume-ath` viene aggiornata.

La procedura calcola poi il rendimento reale di Bitcoin:

$$r_{BTC}(t) = \frac{P_{BTC}(t) - P_{BTC}(t-1)}{P_{BTC}(t-1)}.$$

Nel codice, questa quantità è memorizzata in `btc-return`:

```
set btc-return (new-price - btc-prev-price) / btc-prev-price.
```

La volatilità reale viene invece approssimata tramite il valore assoluto del rendimento:

$$\sigma_{BTC}(t) = |r_{BTC}(t)|.$$

Se il prezzo precedente non è positivo, rendimento e volatilità vengono posti uguali a zero, evitando divisioni non valide.

Dopo il calcolo del rendimento, il codice aggiorna il prezzo precedente e il prezzo corrente:

```
set btc-prev-price btc-price,    set btc-price new-price.
```

In questo modo il modello mantiene memoria del prezzo usato nel passo temporale precedente e registra il nuovo prezzo reale appena letto dalla serie storica.

Se `btc-initial-price` non è ancora stato assegnato, il primo prezzo positivo viene usato sia come prezzo iniziale sia come massimo iniziale:

```
btc-initial-price = btc-price,    btc-ath-price = btc-price.
```

Infine, se il nuovo prezzo supera il massimo storico registrato, viene aggiornato `btc-ath-price`. La sigla ATH indica *All-Time High*, cioè il massimo prezzo raggiunto nella finestra di simulazione.

La procedura termina incrementando `day-index` di una unità. In questo modo, alla chiamata successiva, il modello leggerà il record temporale seguente della lista `prices`.

0.20 Correlazione dei volumi e switch automatico di regime

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; CALCOLO E PLOT CORRELAZIONE VOLUMI (PEARSON)
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to aggiorna-storico-volumi
6
7   set model-volume-history lput model-trading-volume-usd model-volume-history
8   set real-volume-history lput real-trading-volume-usd real-volume-history
9
```

```

10 ;; Applicazione finestra mobile agli storici dei volumi
11 if is-number? window-size and window-size > 0 [
12   if length model-volume-history > window-size [
13     set model-volume-history but-first model-volume-history
14     set real-volume-history but-first real-volume-history
15   ]
16 ]
17
18 ;; Calcolo correlazione di Pearson per i volumi
19 if length model-volume-history >= 2 [
20   let mean-m mean model-volume-history
21   let mean-r mean real-volume-history
22
23   let cov sum (map [ [m r] -> (m - mean-m) * (r - mean-r) ] model-volume-history
24     real-volume-history)
25   let std-m sqrt sum (map [ m -> (m - mean-m) ^ 2 ] model-volume-history)
26   let std-r sqrt sum (map [ r -> (r - mean-r) ^ 2 ] real-volume-history)
27
28   ifelse (std-m > 0 and std-r > 0) [
29     set volume-correlation cov / (std-m * std-r)
30   ] [
31     set volume-correlation 0
32   ]
33
34   ;; Applicazione finestra mobile allo storico della correlazione
35   set volume-correlation-history lput volume-correlation volume-correlation-
36     history
37   if length volume-correlation-history > window-size [
38     set volume-correlation-history but-first volume-correlation-history
39   ]
40
41   set mean-volume-correlation mean volume-correlation-history
42 ]
43
44 if (not current-disabled?) and (length model-volume-history >= window-size) [
45   ifelse mean-volume-correlation >= pearson-threshold [
46     set above-threshold-counter above-threshold-counter + 1
47   ] [
48     set above-threshold-counter 0
49   ]
50
51   if above-threshold-counter > 5 [
52     set current-disabled? true
53     print (word "!!! REGIME RAGGIUNTO A GIORNO " day-index " !!!")
54     print (word "Correlazione media volumi (" precision mean-volume-correlation 3
55       ") >= " pearson-threshold " per piu' di 5 tick consecutivi.")
56     print "Passaggio a FASE 2 (Prezzo Endogeno).")
57   ]
58 ]
59
60 end
61
62 to aggiorna-plot-correlazione-volumi
63   carefully [
64     set-current-plot "Correlazione Volumi (Pearson r)"
65
66     assicura-penna "soglia-switch" red
67     plotxy day-index pearson-threshold
68
69     assicura-penna "zero" black
70     plotxy day-index 0
71
72     if real-trading-volume-usd > 0 [

```

```

72     assicura-penna "correlazione" blue
73     plotxy day-index volume-correlation
74
75     assicura-penna "correlazione-media" green
76     plotxy day-index mean-volume-correlation
77 ]
78 ] [ ]
79
80 end

```

Listing 20: Calcolo della correlazione di Pearson sui volumi e aggiornamento del relativo plot

La procedura **aggiorna-storico-volumi** aggiorna gli storici dei volumi generati dal modello e dei volumi reali. A ogni chiamata, il valore corrente del volume simulato in dollari viene aggiunto a **model-volume-history**, mentre il volume reale viene aggiunto a **real-volume-history**.

Per evitare che gli storici crescano indefinitamente, il codice applica una finestra mobile di ampiezza **window-size**. Se la lunghezza degli storici supera tale valore, viene eliminato il primo elemento tramite **but-first**. In questo modo, la correlazione viene calcolata solo sugli ultimi valori osservati, non sull'intera storia della simulazione.

Quando sono disponibili almeno due osservazioni, il codice calcola la correlazione di Pearson tra la serie dei volumi simulati e la serie dei volumi reali. Vengono prima calcolate le medie:

$$\bar{V}_m = \text{mean model-volume-history}, \quad \bar{V}_r = \text{mean real-volume-history}.$$

Successivamente vengono calcolate la covarianza non normalizzata e le deviazioni standard non normalizzate:

$$\text{cov} = \sum_i (V_{m,i} - \bar{V}_m) (V_{r,i} - \bar{V}_r),$$

$$\sigma_m = \sqrt{\sum_i (V_{m,i} - \bar{V}_m)^2}, \quad \sigma_r = \sqrt{\sum_i (V_{r,i} - \bar{V}_r)^2}.$$

La correlazione viene quindi calcolata come

$$r_V = \frac{\text{cov}}{\sigma_m \sigma_r}.$$

Se almeno una delle due deviazioni standard è nulla, la correlazione viene posta uguale a 0, perché non è possibile misurare una relazione lineare quando una delle due serie è costante.

Il valore ottenuto viene salvato in **volume-correlation** e aggiunto allo storico **volume-correlation-history**. Anche questo storico viene mantenuto entro la dimensione massima **window-size**. La variabile **mean-volume-correlation** rappresenta la media recente delle correlazioni dei volumi.

La parte finale della procedura gestisce lo switch automatico tra Fase 1 e Fase 2. Finché **current-disabled?** è falso, il modello resta nella fase guidata dai dati reali. Quando lo storico dei volumi ha raggiunto la lunghezza della finestra mobile, il codice controlla se

$$\text{mean-volume-correlation} \geq \text{pearson-threshold}.$$

Se la condizione è verificata, il contatore **above-threshold-counter** aumenta di 1. Se invece la correlazione media scende sotto soglia, il contatore viene azzerato. Quando il contatore supera 5, il modello considera raggiunto un regime sufficientemente stabile e imposta

$$\text{current-disabled?} = \text{true}.$$

Questo provoca il passaggio alla Fase 2, in cui il prezzo non viene più copiato dal mercato reale ma viene generato endogenamente dal modello.

La procedura **aggiorna-plot-correlazione-volumi** aggiorna il grafico **Correlazione Volumi (Pearson r)**. Il blocco **carefully** evita che eventuali problemi con il plot o con le penne blocchino la simulazione.

Nel grafico vengono tracciate quattro curve o riferimenti:

$$\text{soglia-switch}, \quad \text{zero}, \quad \text{correlazione}, \quad \text{correlazione-media}.$$

La penna **soglia-switch** disegna la soglia **pearson-threshold**, mentre la penna **zero** disegna la linea di riferimento $r = 0$. Se il volume reale è positivo, vengono poi tracciate la correlazione istantanea dei volumi e la sua media mobile. In questo modo il plot mostra sia il valore corrente della correlazione sia la sua tendenza recente, rendendo visibile il momento in cui il modello si avvicina alla soglia di switch.

0.21 Correlazione di Pearson tra prezzi reali e prezzi simulati

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; CALCOLO E PLOT CORRELAZIONE PREZZI (PEARSON)
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to aggiorna-correlazione-prezzi
6
7   set model-price-history lput model-price model-price-history
8   set real-price-history lput btc-price real-price-history
9
10  ;; Finestra mobile sugli storici dei prezzi
11  if is-number? window-size and window-size > 0 [
12    if length model-price-history > window-size [
13      set model-price-history but-first model-price-history
14      set real-price-history but-first real-price-history
15    ]
16  ]
17
18  ;; Calcolo correlazione di Pearson per i prezzi
19  if length model-price-history >= 2 [
20    let mean-m mean model-price-history
21    let mean-r mean real-price-history
22
23    let cov sum (map [ [m r] -> (m - mean-m) * (r - mean-r) ] model-price-history
24                  real-price-history)
25    let std-m sqrt sum (map [ m -> (m - mean-m) ^ 2 ] model-price-history)
26    let std-r sqrt sum (map [ r -> (r - mean-r) ^ 2 ] real-price-history)
27
28    ifelse (std-m > 0 and std-r > 0) [
29      set price-correlation cov / (std-m * std-r)
30    ] [
31      set price-correlation 0
32    ]
33
34    ;; Finestra mobile sullo storico della correlazione
35    set price-correlation-history lput price-correlation price-correlation-history
36    if length price-correlation-history > window-size [
37      set price-correlation-history but-first price-correlation-history
38    ]
39
40    set mean-price-correlation mean price-correlation-history
41  ]
42 end
43
44 to aggiorna-plot-correlazione-prezzi
45
46  carefully [
47    set-current-plot "Correlazione Prezzi (Pearson r)"
48
49    assicura-penna "zero" black
50    plotxy day-index 0
51
52    if btc-price > 0 [
53      assicura-penna "correlazione" orange
54      plotxy day-index price-correlation
55
56      assicura-penna "correlazione-media" magenta
57      plotxy day-index mean-price-correlation
58    ]
59  ] [ ]
60
61 end
```

Listing 21: Calcolo della correlazione di Pearson sui prezzi e aggiornamento del relativo plot

La procedura **aggiorna-correlazione-prezzi** confronta l'andamento del prezzo generato dal modello con quello del prezzo reale di Bitcoin. A ogni aggiornamento, il prezzo endogeno **model-price** viene aggiunto allo storico

model-price-history,

mentre il prezzo reale **btc-price** viene aggiunto allo storico

real-price-history.

Anche in questo caso viene usata una finestra mobile di ampiezza **window-size**. Se gli storici superano questa lunghezza, il primo elemento viene eliminato tramite **but-first**. In questo modo la correlazione misura la somiglianza recente tra le due serie, invece di dipendere da tutta la storia precedente della simulazione.

Quando sono disponibili almeno due osservazioni, il codice calcola la correlazione di Pearson tra prezzi simulati e prezzi reali. Prima vengono calcolate le medie:

$$\bar{P}_m = \text{mean model-price-history}, \quad \bar{P}_r = \text{mean real-price-history}.$$

Poi vengono calcolate la covarianza non normalizzata e le due deviazioni standard:

$$\text{cov} = \sum_i (P_{m,i} - \bar{P}_m)(P_{r,i} - \bar{P}_r),$$
$$\sigma_m = \sqrt{\sum_i (P_{m,i} - \bar{P}_m)^2}, \quad \sigma_r = \sqrt{\sum_i (P_{r,i} - \bar{P}_r)^2}.$$

La correlazione dei prezzi è quindi

$$r_P = \frac{\text{cov}}{\sigma_m \sigma_r}.$$

Se una delle due deviazioni standard è nulla, la correlazione viene posta uguale a 0. Questo evita divisioni per zero e indica che non è possibile valutare una relazione lineare significativa se una delle due serie è costante. Il valore ottenuto viene salvato in **price-correlation** e aggiunto allo storico **price-correlation-history**. Anche questo storico viene mantenuto entro la finestra mobile. La variabile **mean-price-correlation** contiene la media recente delle correlazioni dei prezzi.

La procedura **aggiorna-plot-correlazione-prezzi** aggiorna il grafico **Correlazione Prezzi (Pearson r)**. Il blocco **carefully** impedisce che eventuali problemi con il plot interrompano la simulazione.

Nel grafico vengono tracciati tre elementi principali:

zero, correlazione, correlazione-media.

La penna **zero** disegna la linea di riferimento $r = 0$. Se il prezzo reale è positivo, vengono poi disegnate la correlazione istantanea tra prezzi e la sua media mobile. Il grafico permette quindi di osservare quanto il prezzo endogeno del modello segua, almeno linearmente, il prezzo reale di Bitcoin.

0.22 Plot di prezzo, volume e parametri d'ordine

```
1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; PLOT PREZZO E VOLUME
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to aggiorna-plot-prezzi-e-volume
6
7   carefully [
8     set-current-plot "Prezzo BTC (USD)"
9
10    assicura-penna "prezzo-reale" blue
11    plotxy day-index btc-price
12
13    assicura-penna "prezzo-stimato" red
```

```

14     plotxy day-index model-price
15
16     set-current-plot "Volume BTC (USD)"
17
18     assicura-penna "modello-usd" green
19     plotxy day-index model-trading-volume-usd
20
21     if real-trading-volume-usd > 0 [
22         assicura-penna "reale-usd" gray
23         plotxy day-index real-trading-volume-usd
24     ]
25 ] [ ]
26
27 end
28
29
30 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
31 ;; PARAMETRI D'ORDINE
32 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
33
34 to-report order-parameter
35     report order-parameter-of turtles
36 end
37
38 to-report order-parameter-a
39     report order-parameter-of agents-a
40 end
41
42 to-report order-parameter-b
43     report order-parameter-of agents-b
44 end
45
46 to-report order-parameter-c
47     report order-parameter-of agents-c
48 end
49
50 to-report order-parameter-of [agenti-selezionati]
51
52     if count agenti-selezionati = 0 [ report 0 ]
53
54     let sx sum [dx] of agenti-selezionati
55     let sy sum [dy] of agenti-selezionati
56
57     report (sqrt (sx ^ 2 + sy ^ 2)) / count agenti-selezionati
58
59 end
60
61
62 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
63 ;; PLOT PARAMETRI D'ORDINE
64 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
65
66 to aggiorna-plot-parametri
67
68     carefully [
69         set-current-plot "Parametri d'ordine"
70
71         assicura-penna "globale" black
72         plotxy ticks order-parameter
73
74         assicura-penna "specie-a" yellow
75         plotxy ticks order-parameter-a
76
77         assicura-penna "specie-b" red
78         plotxy ticks order-parameter-b

```

```

79
80     assicura-penna "specie-c" cyan
81     plotxy ticks order-parameter-c
82 ] [ ]
83
84 end

```

Listing 22: Aggiornamento dei plot e calcolo dei parametri d'ordine

La procedura **aggiorna-plot-prezzi-e-volume** aggiorna i grafici relativi al prezzo e al volume. Il blocco è racchiuso in **carefully**, così eventuali problemi con i plot o con le penne non interrompono la simulazione. Nel plot

"Prezzo BTC (USD)"

vengono tracciate due serie: il prezzo reale di Bitcoin, **btc-price**, e il prezzo stimato dal modello, **model-price**. La prima serie rappresenta il dato esterno o storico, mentre la seconda rappresenta il prezzo generato dalla dinamica degli agenti.

Nel plot

"Volume BTC (USD)"

viene invece tracciato il volume simulato in dollari, dato da **model-trading-volume-usd**. Se il volume reale è disponibile e positivo, viene aggiunta anche la serie **real-trading-volume-usd**. In questo modo è possibile confrontare l'attività di mercato generata dal modello con quella osservata nei dati reali.

La seconda parte del codice introduce i parametri d'ordine. Il parametro d'ordine misura il grado di allineamento collettivo degli agenti. Il reporter generale

order-parameter

calcola il valore sull'intera popolazione, mentre i reporter **order-parameter-a**, **order-parameter-b** e **order-parameter-c** lo calcolano separatamente per le tre specie.

Il calcolo effettivo è svolto da

order-parameter-of [agenti-selezionati].

Se l'insieme di agenti selezionato è vuoto, il reporter restituisce 0. In caso contrario, vengono sommate le componenti direzionali degli agenti:

$$S_x = \sum_i dx_i, \quad S_y = \sum_i dy_i.$$

Il parametro d'ordine è quindi

$$\Phi = \frac{\sqrt{S_x^2 + S_y^2}}{N},$$

dove N è il numero di agenti considerati. Se gli agenti sono orientati in direzioni casuali, le componenti tendono a compensarsi e Φ è vicino a 0. Se invece gli agenti sono fortemente allineati, il modulo della somma vettoriale cresce e Φ si avvicina a 1.

Infine, la procedura **aggiorna-plot-parametri** aggiorna il grafico

"Parametri d'ordine".

Nel plot vengono tracciati il parametro globale e i tre parametri specifici delle specie A, B e C. Questo consente di osservare se l'allineamento emerge in tutta la popolazione oppure solo all'interno di alcuni gruppi.

0.23 Reporter di portafoglio aggregato

```

1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; PORTAFOGLIO
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to-report total-bitcoin
6    report sum [bitcoin-held] of turtles

```

```

7 end
8
9 to-report total-cash
10   report sum [cash-balance] of turtles
11 end
12
13 to-report total-wealth
14   let current-p model-price
15   if current-p <= 0 [ report total-cash ]
16   report total-cash + (total-bitcoin * current-p)
17 end
18
19 to-report bitcoin-a
20   report sum [bitcoin-held] of agents-a
21 end
22
23 to-report bitcoin-b
24   report sum [bitcoin-held] of agents-b
25 end
26
27 to-report bitcoin-c
28   report sum [bitcoin-held] of agents-c
29 end

```

Listing 23: Reporter per Bitcoin, liquidità e ricchezza complessiva degli agenti

Questo blocco contiene alcuni reporter utili per misurare lo stato patrimoniale complessivo degli agenti. Il reporter `total-bitcoin` calcola la quantità totale di Bitcoin detenuta da tutti gli agenti:

$$B_{\text{tot}} = \sum_i B_i,$$

dove B_i rappresenta la quantità di Bitcoin posseduta dall'agente i .

Il reporter `total-cash` calcola invece la liquidità complessiva del sistema:

$$C_{\text{tot}} = \sum_i C_i,$$

dove C_i è il saldo monetario dell'agente i .

Il reporter `total-wealth` restituisce la ricchezza totale degli agenti, sommando la liquidità disponibile al valore in dollari dei Bitcoin posseduti:

$$W_{\text{tot}} = C_{\text{tot}} + B_{\text{tot}}P_m,$$

dove P_m è il prezzo endogeno del modello. Nel codice, tale prezzo è memorizzato nella variabile locale `current-p`. Se il prezzo non è positivo, il reporter restituisce soltanto la liquidità totale, evitando di valorizzare i Bitcoin con un prezzo nullo o negativo.

Gli ultimi tre reporter calcolano la quantità totale di Bitcoin detenuta separatamente dalle tre specie:

$$B_A = \sum_{i \in A} B_i, \quad B_B = \sum_{i \in B} B_i, \quad B_C = \sum_{i \in C} B_i.$$

Nel codice questi valori corrispondono rispettivamente a `bitcoin-a`, `bitcoin-b` e `bitcoin-c`. Queste grandezze permettono di confrontare la distribuzione dell'attività finanziaria tra i diversi gruppi di agenti.

0.24 Flocking dinamico con stubbornness variabile

```

1 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2 ;; FLOCKING DINAMICO
3 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5 to flock
6
7   if any? flockmates [
8
9     let eff-stubbornness dynamic-stubbornness

```

```

10
11   let my-dx cos heading
12   let my-dy sin heading
13
14   let average-dx mean [cos heading] of flockmates
15   let average-dy mean [sin heading] of flockmates
16
17   let weighted-dx (eff-stubbornness * my-dx) + ((1 - eff-stubbornness) * average-
18     dx)
19   let weighted-dy (eff-stubbornness * my-dy) + ((1 - eff-stubbornness) * average-
20     dy)
21
22   if weighted-dx != 0 or weighted-dy != 0 [
23     set heading atan weighted-dx weighted-dy
24   ]
25
26   let speed sqrt (velocity-x ^ 2 + velocity-y ^ 2)
27
28   if speed > 0 [
29     set velocity-x speed * cos heading
30     set velocity-y speed * sin heading
31   ]
32 ]
end

```

Listing 24: Procedura di flocking dinamico degli agenti

La procedura **flock** aggiorna la direzione di ogni agente sulla base dei suoi **flockmates**, cioè degli agenti che compongono la sua rete di riferimento. Il blocco viene eseguito solo se l'agente possiede almeno un vicino:

```
if any? flockmates [ ... ].
```

Il primo valore calcolato è

```
eff-stubbornness = dynamic-stubbornness.
```

Questa quantità misura quanto l'agente tende a mantenere la propria direzione precedente. Una stubbornness alta rende l'agente più conservativo; una stubbornness bassa lo rende più sensibile alla direzione media dei suoi **flockmates**.

La direzione attuale dell'agente viene trasformata nelle sue componenti vettoriali:

$$d_x = \cos(\theta), \quad d_y = \sin(\theta),$$

dove θ è il valore corrente di **heading**. Il codice calcola poi anche la direzione media dei **flockmates**:

$$\bar{d}_x = \langle \cos(\theta_j) \rangle, \quad \bar{d}_y = \langle \sin(\theta_j) \rangle.$$

Il nuovo orientamento è ottenuto come combinazione pesata tra la direzione individuale dell'agente e la direzione media del gruppo:

$$d_x^{\text{new}} = s d_x + (1 - s) \bar{d}_x,$$

$$d_y^{\text{new}} = s d_y + (1 - s) \bar{d}_y,$$

dove s rappresenta la stubbornness efficace. Nel codice queste due componenti sono **weighted-dx** e **weighted-dy**.

Se il vettore risultante non è nullo, il valore di **heading** viene aggiornato tramite

```
set heading atan weighted-dx weighted-dy.
```

In questo modo l'agente non copia semplicemente la direzione media dei vicini, ma si orienta verso una direzione intermedia tra il proprio stato precedente e lo stato collettivo del gruppo.

Dopo aver aggiornato la direzione, la procedura calcola il modulo della velocità:

$$v = \sqrt{\text{velocity-x}^2 + \text{velocity-y}^2}.$$

Se la velocità è positiva, le componenti **velocity-x** e **velocity-y** vengono ricostruite usando lo stesso modulo v , ma con la nuova direzione:

$$v_x = v \cos(\theta), \quad v_y = v \sin(\theta).$$

Questa scelta modifica l'orientamento del moto senza alterare direttamente la velocità scalare dell'agente. La procedura realizza quindi un meccanismo di allineamento dinamico: gli agenti tendono a coordinarsi con la propria rete, ma la forza dell'allineamento dipende dallo stato di stubbornness, che a sua volta può variare con l'hype di mercato.

0.25 Medie della posizione verticale delle specie

```

1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; MEDIE DI POSIZIONE Y
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to-report mean-y-a
6    if count agents-a = 0 [ report 0 ]
7    report mean [ycor] of agents-a
8  end
9
10 to-report mean-y-b
11   if count agents-b = 0 [ report 0 ]
12   report mean [ycor] of agents-b
13 end

```

Listing 25: Reporter per la posizione media lungo l'asse y delle specie A e B

Questi due reporter calcolano la posizione media lungo l'asse verticale delle specie **agents-a** e **agents-b**. La coordinata y è rilevante perché, nel modello, distingue la zona associata all'acquisto da quella associata alla vendita.

Il reporter **mean-y-a** restituisce

$$\bar{y}_A = \frac{1}{N_A} \sum_{i \in A} y_i,$$

dove N_A è il numero di agenti della specie A e y_i è la coordinata verticale dell'agente i . Prima del calcolo, il codice controlla che la specie A non sia vuota:

```
if count agents-a = 0 [ report 0 ].
```

Questo evita di calcolare una media su un insieme privo di agenti.

In modo analogo, il reporter **mean-y-b** calcola

$$\bar{y}_B = \frac{1}{N_B} \sum_{i \in B} y_i.$$

Queste quantità permettono di monitorare la posizione media delle due specie nello spazio comportamentale. Se la media è positiva, la specie si trova prevalentemente nella regione associata all'acquisto; se è negativa, si trova prevalentemente nella regione associata alla vendita.

Il confronto tra $\bar{y}_A$ e $\bar{y}_B$ consente quindi di osservare se le due popolazioni tendono a muoversi nella stessa direzione oppure se sviluppano comportamenti opposti.

0.26 Stima della pendenza tramite regressione lineare

```

1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; REGRESSIONE LINEARE
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to-report linear-regression-slope [ ys ]
6

```

```

7  let n length ys
8  if n < 2 [ report 0 ]
9
10 let xs n-values n [ i -> i ]
11 let mean-x mean xs
12 let mean-y mean ys
13
14 let num sum (map [ [x y] -> (x - mean-x) * (y - mean-y) ] xs ys)
15 let den sum (map [ x -> (x - mean-x) ^ 2 ] xs)
16
17 if den = 0 [ report 0 ]
18
19 report num / den
20
21 end

```

Listing 26: Reporter per il calcolo della pendenza di una regressione lineare

Il reporter `linear-regression-slope` calcola la pendenza della retta di regressione lineare associata a una serie di valori `ys`. L'obiettivo è stimare se una quantità osservata nel tempo tende mediamente ad aumentare, diminuire oppure rimanere stabile.

La lista `ys` contiene i valori osservati, mentre le ascisse vengono generate automaticamente tramite

```
let xs n-values n [ i -> i ].
```

In questo modo, a una lista di n valori

$$y_0, y_1, \dots, y_{n-1}$$

viene associata la sequenza temporale

$$x_0 = 0, \quad x_1 = 1, \quad \dots, \quad x_{n-1} = n - 1.$$

Se la lista contiene meno di due valori, il reporter restituisce 0, perché non è possibile stimare una pendenza significativa con un solo punto.

Il codice calcola poi le medie delle ascisse e delle ordinate:

$$\bar{x} = \frac{1}{n} \sum_i x_i, \quad \bar{y} = \frac{1}{n} \sum_i y_i.$$

La pendenza della regressione lineare viene calcolata come

$$b = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}.$$

Nel codice, il numeratore è memorizzato nella variabile `num`, mentre il denominatore è memorizzato in `den`. Se `den` è nullo, il reporter restituisce 0, evitando una divisione non valida.

Il valore finale

```
report num / den
```

rappresenta quindi la pendenza stimata. Se la pendenza è positiva, la serie `ys` mostra una tendenza crescente; se è negativa, mostra una tendenza decrescente; se è vicina a zero, non presenta una tendenza lineare marcata.

0.27 Correlazione di Pearson generica

```

1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; CORRELAZIONE DI PEARSON (GENERICA)
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to-report pearson-correlation [ xs ys ]
6
7      let n length xs
8      if n < 2 [ report 0 ]
9
10     let mean-x mean xs

```

```

11  let mean-y mean ys
12
13  let cov sum (map [ [x y] -> (x - mean-x) * (y - mean-y) ] xs ys)
14  let sx sqrt sum (map [ x -> (x - mean-x) ^ 2 ] xs)
15  let sy sqrt sum (map [ y -> (y - mean-y) ^ 2 ] ys)
16
17  if (sx = 0) or (sy = 0) [ report 0 ]
18
19  report cov / (sx * sy)
20
21 end

```

Listing 27: Reporter generico per il calcolo della correlazione di Pearson

Il reporter `pearson-correlation` calcola la correlazione di Pearson tra due liste numeriche, indicate come `xs` e `ys`. Questo reporter è generico, quindi può essere riutilizzato per confrontare serie temporali diverse, come prezzi, volumi, parametri d'ordine o altre grandezze dinamiche del modello.

La variabile

$$n = \text{length } xs$$

rappresenta il numero di osservazioni considerate. Se la lista contiene meno di due elementi, il reporter restituisce 0, perché non è possibile calcolare una correlazione significativa con un solo dato.

Vengono poi calcolate le medie delle due serie:

$$\bar{x} = \text{mean } xs, \quad \bar{y} = \text{mean } ys.$$

La quantità `cov` rappresenta la covarianza non normalizzata tra le due serie:

$$\text{cov} = \sum_i (x_i - \bar{x})(y_i - \bar{y}).$$

Le quantità `sx` e `sy` rappresentano invece le deviazioni standard non normalizzate:

$$s_x = \sqrt{\sum_i (x_i - \bar{x})^2}, \quad s_y = \sqrt{\sum_i (y_i - \bar{y})^2}.$$

Se una delle due serie è costante, allora una delle due deviazioni standard è uguale a zero. In quel caso il reporter restituisce 0, evitando una divisione per zero:

```
if (sx = 0) or (sy = 0) [ report 0 ].
```

Se entrambe le serie hanno variazione non nulla, la correlazione di Pearson viene calcolata come

$$r = \frac{\text{cov}}{s_x s_y}.$$

Il valore restituito è compreso tra -1 e 1 . Un valore vicino a 1 indica una relazione lineare positiva, un valore vicino a -1 indica una relazione lineare negativa, mentre un valore vicino a 0 indica assenza di una relazione lineare evidente.

Nel modello, questa procedura è utile perché permette di misurare in modo uniforme quanto due grandezze evolvano insieme nel tempo, senza dover riscrivere ogni volta il calcolo della correlazione.

0.28 Indice di eterogeneità della popolazione

```

1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; INDICE DI ETEROGENEITA '
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to-report heterogeneity-index
6
7  let tot population-a + population-b + population-c
8  if tot = 0 [ report 0 ]
9
10 let sa population-a / tot
11 let sb population-b / tot

```

```

12 let sc population-c / tot
13
14 report (sa ^ 2) + (sb ^ 2) + (sc ^ 2)
15
16 end

```

Listing 28: Reporter per l'indice di eterogeneità delle specie

Il reporter `heterogeneity-index` calcola un indice basato sulla distribuzione relativa delle tre popolazioni di agenti. La variabile

$$\text{tot} = \text{population-a} + \text{population-b} + \text{population-c}$$

rappresenta il numero complessivo di agenti previsto nel modello. Se questo valore è nullo, il reporter restituisce 0, evitando divisioni non valide.

Successivamente vengono calcolate le quote relative delle tre specie:

$$s_A = \frac{\text{population-a}}{\text{tot}}, \quad s_B = \frac{\text{population-b}}{\text{tot}}, \quad s_C = \frac{\text{population-c}}{\text{tot}}.$$

Il valore finale restituito è

$$H = s_A^2 + s_B^2 + s_C^2.$$

Questo indice misura quanto la popolazione sia concentrata in poche specie. Se le tre specie hanno dimensioni simili, le quote sono distribuite in modo più uniforme e il valore di H è più basso. Se invece una specie domina sulle altre, una delle quote è molto grande e il valore di H aumenta.

Nel caso di tre specie perfettamente bilanciate si ha

$$s_A = s_B = s_C = \frac{1}{3},$$

quindi

$$H = 3 \left(\frac{1}{3} \right)^2 = \frac{1}{3}.$$

Se invece tutta la popolazione appartiene a una sola specie, ad esempio

$$s_A = 1, \quad s_B = s_C = 0,$$

allora

$$H = 1.$$

Quindi il valore dell'indice varia tra $1/3$ e 1 : valori più bassi indicano una popolazione più bilanciata tra le specie, mentre valori più alti indicano una maggiore concentrazione in una singola specie.

0.29 Aggiornamento delle metriche di stabilità

```

1  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
2  ;; AGGIORNAMENTO METRICHE
3  ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
4
5  to aggiorna-metriche-stabilita
6
7    set op-history lput order-parameter op-history
8    if length op-history > window-size [ set op-history but-first op-history ]
9
10   set ya-history lput mean-y-a ya-history
11   if length ya-history > window-size [ set ya-history but-first ya-history ]
12
13   set yb-history lput mean-y-b yb-history
14   if length yb-history > window-size [ set yb-history but-first yb-history ]
15
16   ifelse length op-history >= window-size [
17     let log-op map [ v -> ln (v + 0.0001) ] op-history

```

```

18   set lambda-instability linear-regression-slope log-op
19 ] [
20   set lambda-instability 0
21 ]
22
23   ifelse length ya-history >= window-size [
24     set phase-correlation-ab pearson-correlation ya-history yb-history
25   ] [
26     set phase-correlation-ab 0
27   ]
28
29   set heterogeneity-index-value heterogeneity-index
30
31 end
32
33 to aggiorna-plot-stabilita
34 end

```

Listing 29: Procedura di aggiornamento delle metriche di stabilità del sistema

La procedura `aggiorna-metriche-stabilita` aggiorna alcune grandezze utilizzate per descrivere la stabilità e la struttura collettiva del sistema. Le tre serie storiche principali sono:

`op-history,      ya-history,      yb-history.`

La prima contiene lo storico del parametro d'ordine globale, mentre le altre due contengono le posizioni medie lungo l'asse y delle specie A e B.

A ogni chiamata, il valore corrente del parametro d'ordine viene aggiunto a `op-history`:

`set op-history lput order-parameter op-history.`

In modo analogo, `mean-y-a` e `mean-y-b` vengono aggiunti rispettivamente a `ya-history` e `yb-history`. Per ciascuna lista viene poi applicata una finestra mobile di ampiezza `window-size`: se la lista diventa troppo lunga, il primo elemento viene eliminato con `but-first`. In questo modo le metriche vengono calcolate solo sulla dinamica recente del sistema.

Quando `op-history` ha raggiunto la lunghezza `window-size`, il modello calcola una stima di instabilità:

$$\lambda = \text{slope} [\log(\Phi(t) + 0.0001)],$$

dove $\Phi(t)$ è il parametro d'ordine. Nel codice, la lista logaritmica è costruita tramite

`let log-op map [ v -> ln (v + 0.0001) ] op-history.`

Il termine 0.0001 evita di calcolare il logaritmo di zero. La pendenza della regressione lineare viene poi calcolata con

`linear-regression-slope log-op.`

Il risultato viene salvato in `lambda-instability`. Se questa quantità è positiva, il parametro d'ordine tende a crescere nel tempo; se è negativa, tende a diminuire; se è vicina a zero, il livello di ordine collettivo è approssimativamente stabile.

Quando anche gli storici `ya-history` e `yb-history` hanno raggiunto la lunghezza richiesta, il modello calcola la correlazione di Pearson tra le posizioni medie verticali delle specie A e B:

$$r_{AB} = \text{corr}(\bar{y}_A, \bar{y}_B).$$

Nel codice:

`set phase-correlation-ab pearson-correlation ya-history yb-history.`

Questa metrica misura se le due specie tendono a muoversi verticalmente in modo coordinato. Un valore positivo indica movimenti mediamente concordi, un valore negativo indica movimenti opposti, mentre un valore vicino a zero indica assenza di una relazione lineare evidente.

Infine, il codice aggiorna l'indice di eterogeneità:

`set heterogeneity-index-value heterogeneity-index.`

Questa variabile conserva il valore dell'indice che misura quanto la popolazione sia bilanciata o concentrata tra le tre specie.

La procedura **aggiorna-plot-stabilita** è invece attualmente vuota. Essa può essere interpretata come un placeholder, cioè una procedura predisposta per un eventuale futuro aggiornamento dei grafici relativi alle metriche di stabilità.